LOG ANALYSIS AND FAILURE PREDICTION FOR DATA PIPELINE BATCH JOBS USING XGBOOST ALGORITHM

BY

AJIBOLA, MOJOLAOLUWA DAVID

22CG031820

A PROJECT REPORT SUBMITTED TO THE DEPARTMENT OF COMPUTER AND INFORMATION SCIENCES, COLLEGE OF SCIENCE AND TECHNOLOGY, COVENANT UNIVERSITY OTA, OGUN STATE.

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE BACHELOR OF SCIENCE (HONOURS) DEGREE IN COMPUTER SCIENCE

JUNE 2026

CERTIFICATION

I hereby certify that this project was carried out by Ajibola, Mojolaoluwa David in the department of Computer and Information Sciences, college of Science and Technology, Covenant University, Ota, Ogun State, Nigeria under my supervision.

Dr Iheanetu Olamma

Supervisor **Signature and Date**

Prof. Oni Aderonke

Head of Department **Signature and Date**

ACKNOWLEDGEMENTS

My most profound gratitude goes to Almighty God, who has kept me and sustained me from the very start of my degree until now. To Him be all the glory.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF FIGURES

ABBREVIATIONS

AI - Artificial Intelligence

API - Application Programming Interface

CPU- Central Processing Unit

DT- Decision Trees

ELT- Extract Load Transform

ETL – Extract Transform Load

GB – Gradient Boost

HPC- High Performance Computing

I/O – Input /Output

IOT- Internet of Things

IT – Information Technology

KNN- K-Nearest Neighbor

LSTM – Long Short-Term Memory

ML – Machine Learning

NB- Naïve Bayes

NoSQL – Not Only Structured Query Language

QDA – Quadratic Discriminant Analysis

RNN- Recurrent Neural Network

CHAPTER ONE

INTRODUCTION

1.1. Background Information

In the modern data-oriented environment, the role of organizations is becoming dependent on data pipelines to Extract, Transform, and Load (ETL) large volumes of information on various sources into central repositories to analyze it and make decisions. Data pipelines play a crucial role in today's data-centric age and have become fundamental components in enterprise Information Technology (IT) infrastructures. By collecting, processing, and transferring data, data pipelines make it possible to use the ever-growing amounts of information to gain valuable insights and make improved decisions (Foidl et al., 2024). Modern data engineering thrives on data pipelines, which are automated data collection, data movement, and data transformations processes performed on incoming data to bring them to specific destinations. Enterprise data architecture is based on these pipelines, which are the basis of business intelligence, predictive analytics, and operational reporting (Foidl et al., 2024).

Although classic data pipeline designs remain useful in many batch-oriented environments, they are under increasing pressure to support high-volume, high-velocity, and diverse data streams. Modern pipelines often combine structured and semi-structured sources, changing schemas, cloud storage, scheduled transformations, and downstream analytics dependencies. These conditions make manual monitoring less reliable because a small ingestion, cleaning, or transformation defect can move silently into other stages of the pipeline before the issue becomes visible to users (Chanda, 2024).

Recent studies show that data-related pipeline problems often emerge around type mismatches, cleaning, ingestion, integration, and compatibility issues (Foidl et al., 2024). These findings support the need for automated monitoring and prediction because failure can affect multiple downstream systems before it is discovered by a data engineer. The present study therefore treats machine learning as a support mechanism for early warning, not as a replacement for engineering judgement or sound pipeline design.

Artificial intelligence and machine learning can support pipeline management in areas

such as anomaly detection, error prioritization, schema monitoring, resource awareness, and operational decision support. In this report, the role of machine learning is narrowed to log analysis and batch-job failure prediction so that the implementation remains measurable and defensible. The project does not claim full autonomous pipeline repair; it demonstrates how predictive evidence can be presented to an operator before or during a risky execution state (Seenivasan, 2025).

Machine learning-based log analysis and failure prediction in data pipelines is therefore an important direction in data engineering. Conventional monitoring is often reactive because errors are handled after failure messages, missing outputs, or downstream quality issues appear. A predictive monitoring layer can instead summarize historical and live execution evidence, detect patterns associated with risk, and support proactive intervention before a batch job causes wider disruption (Popovic et al., 2024). Automated ETL research also supports the need for stronger data quality and governance controls within pipeline workflows (Ogunsola et al., 2022).

The goal of this study is to apply XGBoost to ETL and data-pipeline batch-job evidence so that likely failures can be identified before they produce downstream data-quality or reporting problems. The expected contribution is a defensible monitoring workflow that connects log ingestion, feature extraction, model prediction, risk classification, dashboard presentation, and report generation. This closing focus leads directly to the problem addressed in the next section: current monitoring practices often reveal failures only after they have already affected the pipeline.

1.2. Statement of the Problem

The non-observability of data pipeline batch job failures in enterprise settings can allow errors to remain unnoticed until they create production data-quality problems. Manual log analysis keeps data engineering teams in a reactive maintenance position instead of enabling proactive pipeline optimization (Akash & Charate, 2025). Recent data-pipeline quality research also shows that data type, cleaning, ingestion, and compatibility issues remain common sources of pipeline defects, which strengthens the need for earlier automated detection (Foidl et al., 2024).

Existing monitoring systems can often display logs, status messages, or completed failure

alerts, but they do not always learn from historical failure patterns or convert live execution evidence into an early warning score. As a result, engineers may spend substantial time tracing integration and transformation issues after the problem has already affected reporting, analytics, or downstream business decisions (Chanda, 2024; Ogunsola et al., 2022). The gap addressed by this study is therefore the absence of an integrated, machine-learning-supported workflow that can characterize failure patterns, classify risk during execution, and present the evidence in a form that supports timely intervention.

1.3 Aim and Objectives of the Study

The aim of this study is to design and develop a log analysis and failure prediction system for ETL batch jobs using XGBoost. The specific objectives include:

i. to identify and characterize common failure patterns in data pipeline batch jobs.
ii. to carry out a comparative analysis based on accuracy metrics for multiple machine learning algorithms to effectively predict pipeline failures.
iii. to develop a prediction model to continuously monitor pipeline execution of batch jobs.
iv. to test and evaluate the system using the following metrics, Precision, Recall and F1-Score.

to visually represent the monitoring workflow using a dashboard and orchestration evidence.

1.4 Methodology

The development of a log analysis and failure prediction application for data pipeline batch jobs follows a systematic, iterative approach integrating data science, machine learning engineering, and software development methodologies.

The study implements a secondary data collection method, whereby quantitative data relating to the study is gathered from online datasets and data repositories. The study utilizes data relating to pipeline execution logs, system performance metrics and historical failure records to identify and characterize batch job failures. The study will comparatively compare multiple machine learning algorithms for their effectiveness in predicting pipeline failures.

The study uses an agile methodology approach to develop the failure prediction application in small iterations that can continuously monitor pipeline execution for batch jobs. The system is evaluated using Precision, Recall, F1-Score and supporting metrics, and its monitoring workflow is represented visually through the Streamlit dashboard, generated reports, and Airflow DAG evidence.

Table 1.1: Objectives-Methodology Table

S/N	OBJECTIVES	METHODOLOGY
1.	To identify and characterize common failure patterns in data pipeline batch jobs.	Collect pipeline execution logs, workload traces, and ETL demonstration evidence; extract severity, keyword, retry, resource, timing, and task-failure features.
2.	To carry out a comparative analysis based on accuracy metrics for multiple machine learning algorithms to effectively predict pipeline failures.	Compare XGBoost with logistic regression, decision tree, random forest, histogram gradient boosting, and a majority baseline using saved evaluation outputs.
3.	To develop a prediction model to continuously monitor pipeline execution of batch jobs.	Train source-specific prediction artifacts and integrate the selected ETL/HDFS-compatible model with the live runner and predictor plugin.
4.	To test and evaluate the system using Precision, Recall and F1-Score.	Compute confusion-matrix counts and evaluate precision, recall, F1-score, ROC-AUC, PR-AUC, MCC, specificity, log loss, and inference latency.
5.	To visually represent the monitoring workflow through a dashboard and orchestration view.	Develop a Streamlit dashboard and Airflow DAG evidence to present pipeline stages, logs, risk classifications, review controls, model comparisons, and generated reports.

1.5 Significance of the study

This study is relevant across multiple fields of study. It suggests techniques for detecting and predicting errors in data pipelines before they happen. It also provides comprehensive information of the core factors and reasons why a particular batch job failed. Firstly, this research contributes to the growing body of knowledge at the intersection of machine learning and data engineering, an area that has received increasing attention as organizations grapple with the challenges of managing complex data ecosystems. While significant research has focused on optimizing individual components of data pipelines such as transformation efficiency, storage optimization, or query performance relatively less attention has been devoted to holistic, intelligent pipeline management systems capable of learning from operational history and predicting failures (Foidl et al., 2024).

Secondly, this study is underscored by the operational cost of pipeline failure and the burden of maintaining complex data ecosystems. Data-related incidents can affect reporting accuracy, compliance activities, customer-facing services, and downstream analytics (Foidl et al., 2024). The value of the proposed system is therefore not only in model accuracy, but also in the way it converts logs, metrics, and alerts into an interpretable monitoring workflow that helps engineers respond earlier (Ogunsola et al., 2022).

Lastly, the study also contributes to the growing discourse on AI-driven infrastructure management and autonomous systems, demonstrating how machine learning can be applied not just to business analytics but to the fundamental infrastructure that enables those analytics. As organizations continue to grapple with increasing data complexity and velocity, the principles and techniques developed in this research will remain relevant and can be extended to address emerging challenges in real-time streaming pipelines, edge computing scenarios, and distributed data processing environments.

1.6 Scope of the study

The study focuses on presenting the design of machine learning driven framework for predicting failures in batch-oriented data pipeline jobs – ETL pipelines in this context. The paper deals with log-based data sources namely application logs, system logs and database logs for failure pattern detection and characterization.

Several types of pipeline failures such as resource exhaustion failures (memory, CPU, disk space), data quality anomalies (schema inconsistencies, missing values, type mismatch), and dependency failure type failures (upstream failure, connection timeouts), are addressed. Supervised machine learning and ensemble models are explored as methodological approaches. The complete end-to-end predictive pipeline including log data ingestion is covered and the solution is expected to perform well on log-based data in systems where log data is sufficiently abundant, consistent across pipeline components, and where failure events are sufficiently frequent and diverse to enable the development and evaluation of predictive models.

1.7 Project Outline

This study will consist of five comprehensive chapters that will adequately explain the research question. Chapter one consists of the introduction to the research, background information, statement of problem, scope and significance of the study and a little intro into the methodology of the research. Chapter 2 will consist of a very comprehensive literature review of various papers related to the study. It will talk about core concepts of the study and look at already existing systems and their limitations; it will also look at the gaps this research aims to cover.

Chapter three will consist of the methodology; it will include an extensive and deep dive into the specific method and algorithms that the study is implementing to achieve the aims. It will talk about the various datasets the study will utilize Chapter four will comprise of the results and analysis the study was able to achieve, it will show whether the study was successful or not in achieving the goal. Chapter five will consist of the project conclusion and areas where future research can be conducted or improved upon.

CHAPTER TWO

LITERATURE REVIEW

2.1 Preamble

This chapter reviews available literature, and systems relating to failure prediction, log analysis and machine learning methods. It talks about data engineering as a field of study, ETL pipelines and architecture and the evolution of ETL architecture. This chapter also discusses batch jobs, batch job failures, their impacts and costs, machine learning and all its traditional algorithms. It also reviews the gaps in already existing systems and the specific area this study covers.

2.2 Review of Data Engineering

Data engineering is a core discipline in modern data-driven organizations. It focuses on designing and building that allows data to be extracted and processed. This processed data can then be used for analytics, for the training of machine learning models or to make business informed decisions. Data pipelines are the core technology behind this process and they could be in the form of Extract, Transform, Load (ETL) or Extract, Load, Transform (ELT) pipelines.

ETL pipelines are the architectural basis of integration and processing of data in the present-day data-driven companies. These pipelines are the core of data warehousing and business intelligence systems as they enable movement and transformation of data that could have various origins and load to the target destinations where it could be analyzed and be used in a decision-making process.

2.2.1 Evolution of ETL Architecture

Legacy ETL pipelines were typified by a batch processing paradigm rule-based transformations and the necessity that they can be configured manually. They relied heavily on manual interventions which introduced the potential for human error during data extraction, transformation or loading (Ogunsola et al., 2022). Such legacy systems were not particularly good at managing schema changes, and had to be maintained

manually with significant effort, and have not been very flexible to handle real-time data processing.

Modern ETL is much more advanced than the earliest iterations with use of cloud-native platforms real time data integration, which replaces on-premises processing in batch mode. Automated ETL is aimed at reducing the level of manual interferences when designing, managing and enhancing data pipelines (Chanda, 2024). Various factors have contributed to this transformation; the volumes of data are increasing exponentially, real time analytics are required, there is also the spread of a variety of data sources and there is the implementation of cloud computing infrastructure.

2.2.2 Components and Architecture of ETL pipelines

ETL pipelines are built on three basic steps with each carrying a specific role in the process of data integration. The extraction stage entails the mining of such data in various heterogeneous forms of relational databases, Not Only Structured Query Language (NoSQL) stores as well as Application Programming Interfaces (API's), Internet of Things (IOT) devices, flat files, and streaming platforms. This stage supports different data formats, connection protocol, as well as access methods and guarantees data integrity when they are being accessed. The process typically occurs during scheduled intervals when system load is minimal, with the main goal being to gather all necessary data from sources efficiently (Nishanth R. M., 2019).

The transformation phase is the most complicated stage in ETL process because it involves cleaning of the data, validation, normalization, enriching and aggregation of data and applying business logic. Some of the challenges that transformations must conquer are data quality problems like missing values, duplications, inconsistency as well as format inconsistency. Contemporary transformation engines today often include checks on data quality, schema evolution management and inference of data type or types to minimize human activity. This process may involve data deduplication, format standardization, and applying business rules to make the data uniform and suitable for analysis (Nishanth R. M., 2019).

The loading step sends processed data to the target systems, which can be data warehouses, data lakes, machine-learning feature stores, or operational databases. Strategies to load data involves full refresh or incremental refresh and factors like transaction consistency, error management and performance maximization are taken into consideration. Transformation logic has been moved into the target system with the creation of ELT architectures, by using the compute capabilities of modern cloud data warehouses.This phase is typically scheduled to run during non-peak hours to avoid disrupting the performance of the warehouse (Nishanth R. M., 2019).

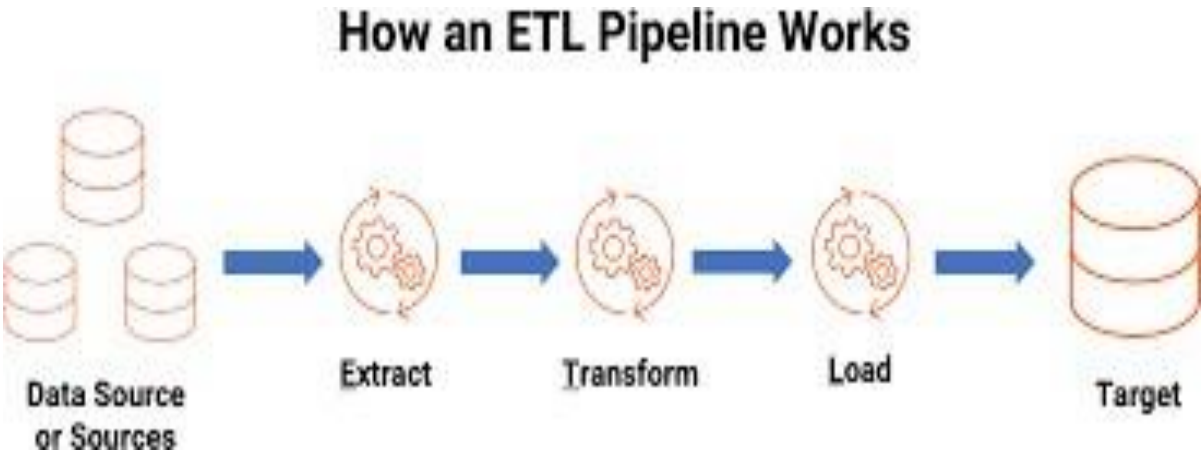

Figure 2.1: A simple representation of an ETL pipeline (Informatica, 2025)

2.3 Review of Batch Job Failures

Enterprise computing is still based on batch job processing, which processes scheduled data processing tasks, generates reports, synchronizes data, performs backup operations, trains machine learning models. It is essential to know how, why, and when batch job failures happen to sustain effective data pipeline processes and business continuity. Analyzed results show that job failure rates are significant in most High-Performance Computers (HPCs), and on average, a failed job often consumes more computational resources than a successful job (Yuan et al., 2012)

The resource usage patterns of failed and completed jobs can differ before termination,

especially where retry activity, task resubmission, scheduling delay, and abnormal consumption appear in the execution history. Large-scale studies of job failures show that failed jobs may consume considerable computation before failure, which makes early detection valuable for cost control and operational reliability (Yuan et al., 2012).

The frequency of failures may also vary across time windows because of resource contention, maintenance activities, dependency failures, and workload changes. Workload analysis in large heterogeneous GPU clusters shows that production traces can contain diverse job types and resource requirements, which supports the use of source-specific models rather than assuming that one feature schema can represent every execution environment (Weng et al., 2022).

2.3.1 Impacts and costs of Failed Batch Jobs

Batch job processing remains useful because large volumes of data can be processed at scheduled intervals, often during periods of lower system demand. The disadvantage is that a failed batch job can delay an entire reporting or analytics workflow, especially when the process runs outside normal working hours. This makes early warning important because the objective is not simply to record that a job failed, but to give engineers a chance to inspect abnormal conditions while the process is still running (Chanda, 2024).

2.3.2 Categories of Extract Load Transform Batch Job Failures

There are several root causes of batch job failures, and they can be classified into broad categories of resource-related, data-related, system-related and configuration-related. The failures associated with resources are because of the exhaustion of memory, CPU, throttling, disk space limitations, bandwidth limits in the network, and even failures in timeouts. Such failures usually happen due to the mistaking of resources obligations or rivalry with other tasks.

Data failures are associated with missing input files, corrupt records, mismatched schema, invalid values, duplicated records, and volumes that exceed expected processing capacity. These failures are challenging because an upstream defect can propagate into transformation, loading, reporting, and machine-learning stages. Data pipeline quality studies therefore emphasize that failure analysis should consider where in the pipeline the

issue occurs and how it affects later processing areas (Foidl et al., 2024).

System failures can include hardware faults, network interruptions, software bugs, platform instability, dependency timeouts, and infrastructure maintenance disruptions. In cloud and cluster environments, the interaction between applications, nodes, schedulers, and resource constraints makes failure prediction difficult because a job may fail for reasons that are only partially visible in application logs (Jassas & Mahmoud, 2022).

Configuration related failures are caused by parameters that are not properly set, unsuitable versions of software, lack of dependencies and inappropriate level of privileges and errors in configuration of schedulers. These failures are often attributed to flaws by people during job definition or environment configuration and hence justification and automated configuration control is necessary.

Machine learning represents a continuum of methods involving allowing systems to learn undirected data without being explicitly programmed. The paradigm is customarily divided into three major paradigms of learning, which apply to various contexts of problems and data access. Machine learning is a part of computer science that emanated from the study of pattern recognition and computational learning theory all in artificial intelligence (Udousoro, 2020). The most popular divisions of machine learning are discussed subsequently.

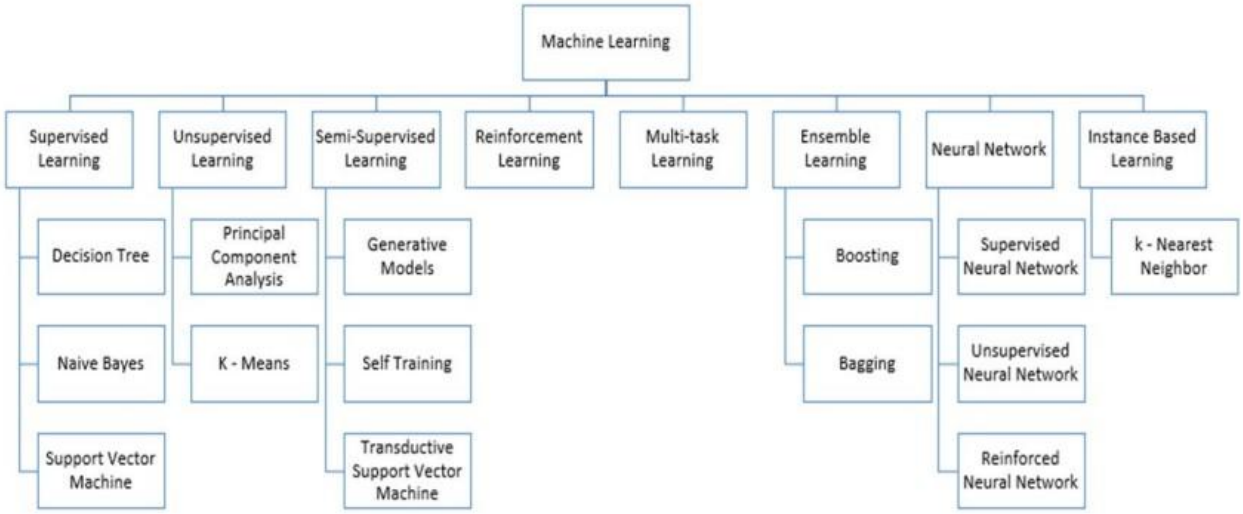

Figure 2.2: Machine learning Techniques

(Udousoro, 2020)

Supervised learning entails the training of models with labeled datasets whereby the input features are accompanied with known output labels. The model will learn to localize the input to the output by reducing errors in prediction on the training data. To anticipate future events, supervised machine learning algorithms use labelled examples to apply what they have learned in the past to fresh data (Shaveta, 2023). The paradigm performs well in cases where the historical labeled data is rich and the dependence between features and results can be acquired with examples. The trainer is required to learn from the training set with examples of labelled set which will be used to identify the unlabeled examples in the test set with highest possible accuracy (Udousoro, 2020). Supervised learning can be grouped further in two categories of algorithms, Regression and Classification algorithms (Shaveta, 2023). Classification algorithms are used to predict discrete values, whereas regression algorithms are used to predict continuous values (Breiman, 2001).

Unsupervised learning also operates on data sets of unspecified labels by exploring patterns, structures or deviations present in the data. This learning technique does not

produce classification but makes decisions that maximize rewards (Udousoro, 2020). The machine is trained using a set of unlabeled, unclassified, or uncategorized data, and the algorithm is required to respond independently to that data (Shaveta, 2023).Unsupervised learning does not need the presence of a person to oversee the model which may be beneficial in some situations. However, supervised learning models provide more accurate results because a programmer explicitly teaches the system what to search for in the data presented Unsupervised learning, on the other hand, may be highly unexpected (Naeem et al., 2023). In the case of log analysis, unsupervised methods are the best when it comes to identifying new failure modes that were not in the training set and unequal system behavior without necessitating massive labeling.

Semi-supervised methods use both labeled and unlabeled data and are useful where labeled failure examples are limited but large quantities of normal logs are available. The main objective of semi-supervised learning is to reduce dependence on fully labeled datasets while still using the structure of available unlabeled data (Reddy et al., 2018). Semi-supervised log-based anomaly detection can be relevant to pipeline monitoring, but its results must be interpreted carefully because the absence of labeled failures can make validation difficult (Yang et al., 2021).

Reinforcement learning trains agents to make sequential decisions by interacting with an environment and receiving rewards or penalties. It is less central to this study than supervised learning because the present project is based on labeled success and failure outcomes rather than online reward optimization. However, reinforcement learning remains relevant to future work because it could support dynamic scheduling, resource allocation, and intervention policies after a failure-prediction system has been validated (Naeem et al., 2020).

2.3.3 Review of Log Analysis using Machine Learning

With increasing amount of logs generated from servers, applications, network devices, security tools, and cloud services in today's computing environment, log analysis becomes a vital task. Traditional log analysis, based on human effort or predefined rules, is inadequate due to the volume, variety, and velocity of log data. Machine learning methods provide a powerful way to automate and improve log analysis, to increase the

ability of detecting anomalies and predicting system failure and to gain more visibility to the system operation.

Machine learning techniques assist in processing structured log data, detecting patterns and deviations that may represent failures, intrusions or performance problems. Unsupervised learning methods such as clustering and anomaly detection models play an important role because unlabeled log data is common. These models will build an understanding of "normal" log patterns and then point out log messages or sequences that do not fit. Supervised learning methods can also be used when labeled logs are available to perform log classification or prediction of particular failure types. Deep learning techniques, particularly Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTMs), and Transformers, have further improved log analysis by modeling sequential dependencies within logs.

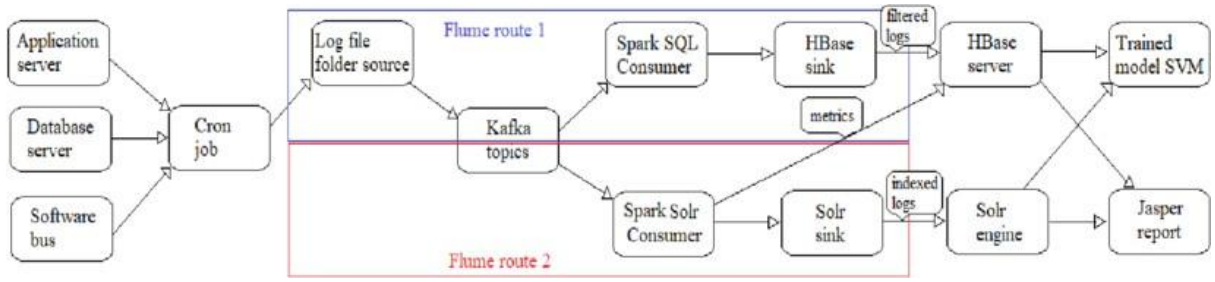

Figure 2.3: Workflow for Log Analysis and Prediction

2.4 Review of Existing Models and Gaps

The knowledge of the current methods of pipeline failure prediction and log-based anomaly detection allows us to give the understanding of the new solution novelty and contributions to the current work. The field of research consists of a wide variety of methodologies consisting of both statistical methods, as well as traditional machine learning and deep learning implementations.

2.4.1 Analysis of Job Failure and Prediction Model for Cloud Computing using Machine Learning

This study focused on predicting failed jobs before they occur to enhance resource consumption and cloud-based efficiency. It also aimed to study various cloud traces to find the correlation between failed and successful jobs. Batch job properties like job priority, scheduling class, resource request was extracted. Three publicly available traces: Google cluster, Mustang and Trinity were used to evaluate the proposed model. The study analyzed both successful and failed jobs using various machine learning algorithms: Decision Trees (DT's), Random Forest (RF), *k*-Nearest Neighbors (KNN), Quadratic Discriminant Analysis (QDA), Gradient Boosting (GB), XGBoost and Naïve Bayes (NB), while applying the proposed model to the three different cloud traces. The proposed model resulted in Decision Trees, Random Forest and XGBoost algorithms achieving the highest accuracy, precision, recall and F1-score. It also proved that jobs requesting too little memory were prone to crashing.

This research thoroughly analyzed job failure prediction in cloud computing using logs, but specific data pipelines or ETL workflows were not accounted for. It also relied heavily on the job metadata without analyzing actual log content or error patterns. This led to the models predicting failures at job submissions, not during the execution when early warnings could have prevented failures.

2.4.2 DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning

DeepLog was the first neural network that applied deep learning to log sequences to detect anomalies in logs with the help of LSTM networks (Du et al., 2017). The system considers logs as natural language patterns and uses LSTM to extract patterns in execution paths. This study proposed a deep neural network model utilizing LSTM, to model system logs as natural language sequences. It implicitly captured the potentially non-linear and high dimensional dependencies among log entries from the training data that correspond to normal system execution paths. The system achieved 88.9% accuracy where top predictions matched actual log keys and 96.1% of predictions were within the top two predictions. DeepLog was deployed in production at Huawei Cloud to analyze terabytes of daily log day using Kafka as a streaming channel or Flink for distributed log

preprocessing.

Long Short-Term Memory models are very extensive in the sense that they need a lot of training data which may not be available for new or evolving pipelines. The system only accounted for general system logs and not specifically optimized batch jobs with their unique properties. This research mainly utilized log text for predicting failures, inconsequentially it ignored CPU, memory Input/Output (I/O) metrics which are also crucial for batch job failures.

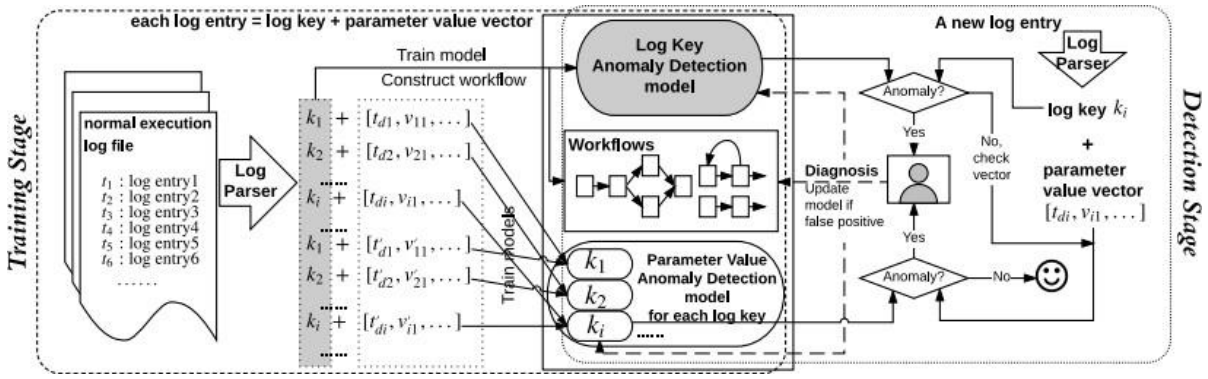

Figure 2.2: DeepLog Architecture (Zou & Petrosian, 2020)

2.4.3 Cloud failure prediction based on traditional machine learning and deep learning This paper made a comprehensive comparison and model evaluation of predictive models for job and task failures. Five machine learning algorithms and three deep learning algorithms were used to train the datasets. A benchmark dataset called Google cloud traces was used for the training and testing of models.

This research established that in the case of job failure prediction, Extreme Gradient Boosting produced the best model where the disk space request and CPU request are the most important features that influence the prediction. It also established that Decision Tree and Random Forest algorithms produce the best models where the priority of the task is the most important feature for both models. Logistic regression model was also discovered to be the most scalable compared to others.

Recent related work has also emphasized that log-anomaly detection must be evaluated from an operational perspective, not only from a model-score perspective. Ma et al. (2024) surveyed

practitioners and reviewed log-anomaly detection studies, showing that adoption depends on whether tools meet practical needs such as trust, usability, and alignment with maintenance workflows. This is relevant to the present study because the Streamlit dashboard, review controls, and generated reports were designed to make prediction evidence usable to an operator rather than leaving it as a raw model output.

Wang et al. (2024) proposed a cross-system log-anomaly detection approach that addresses labeling cost, evolving logs, and adaptation across systems. Their study reinforces the importance of source-specific validation because a model trained in one environment may not automatically generalize to another. This supports the present project's decision to keep ETL/HDFS-compatible, Alibaba, and Google 2019 artifacts separate instead of forcing a single merged model across incompatible schemas.

Zhang et al. (2024) studied multivariate log-based anomaly detection for distributed databases and argued that single-node or single-source evidence may be insufficient in distributed settings. This finding is important for pipeline failure prediction because batch jobs may fail due to interactions among logs, resources, tasks, and dependencies. The present study responds to this by combining log features, workload features, resource context, dashboard evidence, and report generation.

Table 2.1: Summary comparison of selected related works

Study	Dataset/context	Method/focus	Key finding	Limitation/gap addressed by this project
Jassas and Mahmoud (2022)	Cloud-computing job records	Machine-learning job failure prediction	Showed that ML can support job-failure prediction in cloud workloads.	Did not provide the same ETL-facing dashboard, report, and intervention workflow.
Du et al. (2017)	System log sequences	DeepLog with LSTM sequence modeling	Demonstrated deep-learning-based log anomaly detection.	Focused on sequence anomaly detection rather than an attachable ETL batch-job monitoring plugin.
Ma et al. (2024)	Practitioner survey and literature review	Adoption expectations for log anomaly detection	Showed the importance of practical usability and	Supports the need for visual, reviewable prediction evidence in

			trust.	this project.
Wang et al. (2024)	Multiple supercomputing log datasets	Cross-system meta-learning for log anomalies	Addressed adaptation across systems with limited labels.	Reinforces the need for source-specific validation and cautious generalization.
Zhang et al. (2024)	Distributed database logs	Multivariate log-based anomaly detection	Showed that multivariate evidence can improve anomaly detection.	Supports combining logs, resource context, workload features, and reports.

2.5 Gaps Identified in Literature

Although much work has been done on the prediction of job failures in cloud computing and prediction of log anomalies in general software systems, little to no solutions are created to tackle the specifics of the issues in data pipeline batch jobs. Data pipelines have complicated dependencies between jobs, differing data quality demands, diverse data origins, and particular failure strategies in regard to data processing as opposed to merely computational execution. A combined job-level prediction of failures, and log analysis based on data pipeline characteristics is a significant gap that can be addressed.

The current methods of predicting pipeline failures and analyzing the log have serious gaps, although the research on that matter is very large. The majority of research on log anomaly detection tests models on open benchmark datasets which might not be representative of real-world production data pipeline logs in terms of diversity and complexity. Practical pipelines may be based on heterogeneous systems, application specifics, and organization unique settings which vary significantly with standardized benchmark environments.

The capability of real-time prediction is usually absent or of lower priority. Numerous systems of research work on offline analysis or batch detection based on evaluation on historical data. Low-latency prediction is needed by production systems to detect potential failures before they happen, as this would permit the production system to take preventive measures, which puts intense computational efficiency requirements.

CHAPTER THREE

RESEARCH METHODOLOGY

3.1 Preamble

The methodology by which the log analysis and failure prediction system has been developed, demonstrated, and evaluated is described in this chapter. This is primarily a software engineering and machine learning study that seeks to build a working plugin, train context specific models and present a useful visualization of the system as a workflow not a purely command-line system.

The method takes advantage of the clear separation of the 5 layers of evidence: offline training; ETL demonstration pipeline with outputs, predictor plugin with a feature summary report, visualization dashboard and the Apache Airflow DAG representation. These distinct layers serve a dual purpose, demonstrating robustness and making the research defensible: the visual results of the dashboard are convincing on their own, but all this work is supported and verifiable against actual artifacts stored locally or deployed remotely such as saved models, files and code. While synthetic visualization has been used in the demonstration that provides an operator with the visibility of system behavior during the viva, this has been labeled 'demonstration behavior', and not new evidence.

This approach also relies on the practical issue that if the system is to predict failures, the prediction must be able to happen when the effects of the prediction can be used, thus the system is not a standalone batch model. This system provides the operator with visibility of what the system is doing during both the job execution, logging and analysis phases, and not simply presenting the results when the log is finished.

This project relies upon a staged form of evidence; public/local data sets are used to train the model, live logs are used for testing the ETL, a plug-in is used for tangible analysis and report generation and the dashboard is used for both and the report for post analysis. It covers all bases of why batch failure prediction can be as much an operational support challenge as it is a statistical one.

3.2 Research Design

The design of the research is applied experimentally based with a prototype-driven approach to building the software. The experimental aspects of the design involve data loading, feature generation, model training, model comparison, and evaluation on held-out data. The prototype aspects of the design involve executing the pipeline, live attachment, stream analysis, intervention decisions, generating a report, rendering a dashboard, and utilizing Airflow orchestration.

The core predictive method in the constructed system is XGBoost. XGBoost was utilized as a boosting method that can model non-linear relationships between varied operational indicators like number of errors, rate of warnings, amount of retries, count of failed tasks, delay in scheduling, and sampled resources. It was compared with other techniques (logistic regression, decision tree, random forest, histogram gradient boosting, majority baseline) in the project because the defense argument is stronger if the selected model is compared with other techniques.

The design is one of an iterative refinement. Early versions of the system focused on terminal logging and writing log files to storage. Later versions add features like the Streamlit dashboard, live ETL functionality, the predictor plug-in, HIGH-risk review functionality, percentage display as an integer value on the dashboard, model artifacts tailored for the respective sources (Alibaba, Google), and a "ready for deployment" configuration of Streamlit that allows running it on Render. Each iteration attempts to maintain the clear separation between the model-generated evidence and the behavior designed for demonstration.

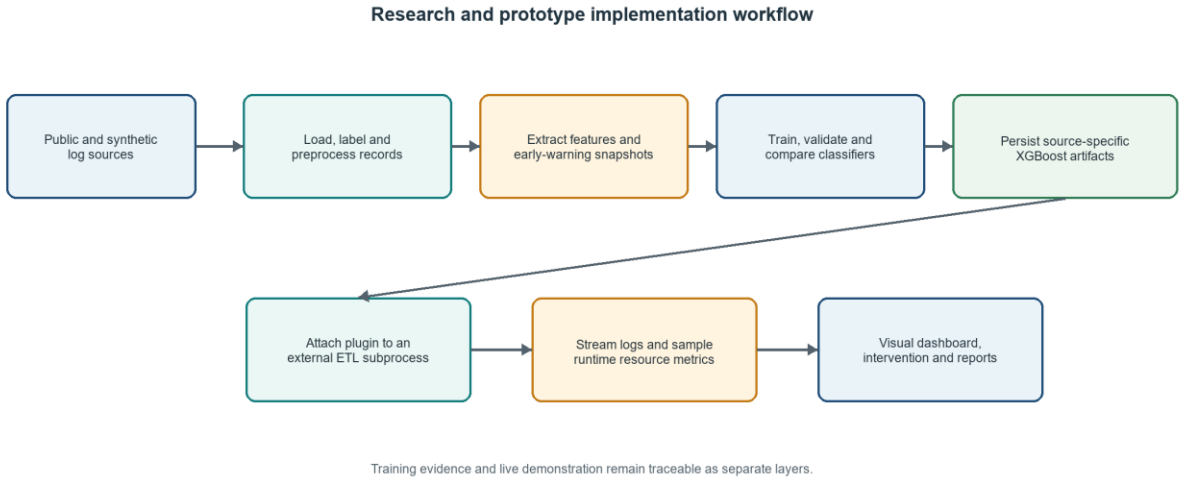

Figure 3.1: Research and prototype implementation workflow

Table 3.1: Methodological alignment with project objectives

Objective	Method used in the implementation	Evidence retained in the project
Identify failure patterns in batch jobs	Log severity counts, keyword signals, retry indicators, failed task counts, resource and scheduling features were extracted from logs and workload traces.	feature_extractor.py; data_loader.py; saved metrics and feature importance charts
Compare machine learning algorithms	A comparison routine evaluated logistic regression, decision tree, random forest, histogram gradient boosting, XGBoost, and a majority baseline.	model_comparison.py; outputs/model_comparison.csv; comparison_alibaba and comparison_google2019 folders
Develop a prediction model for monitoring	Source-specific XGBoost artifacts were trained and saved. The live dashboard uses the ETL/HDFS-compatible artifact for demonstration.	models/xgboost_pipeline_monitor_etl.pkl; models/xgboost_pipeline_monitor_alibaba.pkl; models/xgboost_pipeline_monitor_google2019.pkl
Evaluate with precision, recall and F1	Evaluation files store accuracy, precision, recall, F1, ROC-AUC, PR-AUC, specificity, MCC, log loss, and confusion-matrix counts.	outputs/metrics.json; Chapter Four result tables
Represent the system visually	Streamlit renders stage boards, logs, risk cards, review controls, model comparison charts, reports, and system maps.	dashboard_app.py

The applied experimental part of the design followed a supervised-learning workflow because the available data can be organized into successful and failed outcomes. Supervised learning is suitable when historical labels exist and the objective is to learn a mapping between execution features and a failure outcome. This justified the use of precision, recall, F1-score, ROC-AUC, PR-AUC, confusion-matrix counts, and related metrics, since the research question concerns the

detection of failure risk rather than only the description of log events.

The prototype-development part of the design followed an incremental build-and-test approach. At each stage, a working software capability was added and then checked against the project objectives. The first capability was offline feature extraction and model training. The next was model comparison. Then the predictor plugin, live ETL runner, dashboard, report generator and Airflow DAG work was continued in this order to keep the system designed not just to one screen but one overall monitoring flow.

The choice of XGBoost as the primary model deployed was methodologically conservative as gradient boosted decision tree algorithms have been demonstrated to be excellent with tabular operational features and can model non-linear interactions between counts, rates, timing features, resource variables and failure keywords (Chen & Guestrin, 2016). Review literature also supports the continued relevance of XGBoost in applied machine-learning classification tasks (Arif et al., 2023). The selection was methodologically cautious because XGBoost was not treated as automatically superior. It was compared with logistic regression, decision tree, random forest, histogram gradient boosting, and a majority baseline, so that the final interpretation could distinguish the selected implementation from the broader comparison evidence.

The comparison design also reflects the diversity of machine-learning methods. Logistic regression provides a simple linear baseline (Maalouf, 2011). Decision-tree methods provide interpretable rule-based splits (Quinlan, 1986), and applied predictive work continues to show their usefulness in classification tasks (Matzavela & Alepis, 2021). Recent survey evidence also describes decision trees as a continuing foundation for interpretable machine-learning applications (Mienye & Jere, 2024). Random forests provide ensemble averaging over trees (Breiman, 2001), while support-vector approaches represent margin-based classification traditions (Cortes & Vapnik, 1995). K-nearest neighbor was considered in the wider literature review as a supervised-learning method, but it was not selected for the implemented comparison because the project focused on models better suited to the saved tabular and tree-based workflow (Suyal & Goyal, 2022).

3.3 Population of Study and Data Sources

The populations for this study were the execution records that record whether or not a computation job or pipeline was successful or failed. As there is not just one context that a

pipeline could fail in, multiple sources of evidence have been used throughout this project. The HDFS compatible evidence source, which we are going to use, can allow for log based feature extraction and the live demonstration of ETL, can be supplemented with evidence taken from a Alibaba PAI GPU trace and a Google Borg 2019 trace, which represent evidence in the context of production like workload conditions, or from a synthetic ETL pipeline which is designed to represent predictable execution based on controlled runtime behavior, useful in the live dashboard and plugin demonstrations.

The HDFS-compatible model is used for the live ETL dashboard since the live runner processes incoming pipeline messages and attempts to convert them into an HDFS style stream with a stable job identifier, as neither Alibaba or Google artifacts were shoehorned into this live dashboard's framework and were instead left as separate benchmark artifacts based upon workload trace schema not HDFS log line schema.

The compressed Alibaba and Google trace files were extracted out of the deployable repository prior to rendering deployment preparation, which doesn't impact on the deployed demonstration since the model and demo use persisted model artifacts, small sample data, and persisted comparisons, leaving retraining with larger compressed traces as a research problem that requires only local resources rather than a problem that impacts on deployment.

Table 3.2: Dataset sources and project role

Source	Local role	Why it was used	Deployment status
HDFS-compatible logs	Primary log schema and live ETL-compatible model	Supports log-line grouping, severity counts, failure keywords, and early warning snapshots.	Raw HDFS.log excluded from deployment; trained model retained.
Alibaba PAI GPU trace	Bounded production-trace benchmark	Provides workload-level evidence from task and job records.	Raw archive excluded from deployment; model and comparison summary retained.
Google Borg ClusterData2019	Bounded production-trace benchmark	Provides large-scale cluster workload evidence with a different schema.	Raw archive excluded from deployment; model and comparison summary retained.
Synthetic ETL pipeline	Live defence demonstration	Provides a controllable pipeline with extract, validate, transform, load, and report stages.	Included through source code and sample_orders.csv.

The population was treated as an operational record produced by a computer job or data pipeline execution rather than as human respondents, survey respondents or manually selected observations. The significance is in that unit of analysis which was a job, task, log sequence, or observation window, but never a person. The analysis utilizes records that show execution behavior, the severity level of the task, how much resource it consumed, whether or not it succeeded and whether progress through pipeline stages was observable.

The sources used to augment the evidence set beyond the small ETL demonstration were to represent workload that is representative of what researchers' hope will be analyzed by future machine learning systems. The Alibaba workload trace represented workload of heterogeneous GPU clusters in large jobs (Alibaba Group, 2022). The fact that workload analysis of large heterogeneous GPU clusters should be source-specific supported its use here (Weng et al., 2022). The Google Borg 2019 trace provides production-style workload from a different cluster-configuration and data-schema. Together, the heterogeneous workload traces show the importance of treating workload as potentially source-specific models (Google, 2019). These sources support the methodological decision to retain source-specific models rather than force all observations into one artificial scheme.

The role of the HDFS-compatible data source and ETL demonstration was to show evidence related to log grouping, real-time feature extraction and visible risk movement within the dashboard. While public log collections show the viability of using machine learning with logs (Zhu et al., 2023). he ETL demonstration thus served as an exhibit and bridge between research evidence and examiner-facing system behavior.

3.4 Sampling Technique

A bounded purposive sampling technique was used because the public trace files are large and the project had to remain reproducible on a local development computer. The bounded samples were large enough to train and evaluate the source-specific models while avoiding the deployment problem of committing multi-gigabyte raw logs to GitHub.

The Alibaba loader reads job and task level records and compiles a bounded labeled sample of 5,000 jobs (273 failures and 4,727 successful jobs). Google 2019 loader creates a bounded

sample of 10,000 labeled records (2,299 failures and 7,701 successful jobs). The ETL demonstration data was gathered through a set of scenario loops which yielded 100 ETL records and 400 early-warning snapshots that are stored and made visible for reporting purposes.

It is not claimed that the bounded samples are representative of all data that exist in the traces for Alibaba and Google, but that the sample provides more benchmark data for the prototype. Chapter Five recommends performing larger experiments, but this report only provides the evidence available in the saved artifacts.

Purposive bounded sampling was used, since the aim of the project was to create a defensible prototype (and demonstrate it), rather than analyzing every row from each trace. This makes the experiments viable on a local machine and does not create a deployed repository dependent on multi-gigabyte raw logs. All sample sources are programmed via loaders, so the logic behind selecting sample data can be inspected and reproduced (as opposed to manual selection through spreadsheets).

The sampling approach also considered class imbalance: since failures are typically rarer than successes, the model's accuracy may appear quite good even if the model performs poorly on failures. For this reason, we ensured that the number of failures in each bounded sample were respected and also considered metrics which are more sensitive to minority class: we therefore have accuracy, precision, recall, F1-score, PR-AUC and MCC in the report.

The sample used for the ETL demonstration was selected differently. The aim was not to have a representative sample of all production pipelines; it was to be able to demonstrate repeatable scenarios displaying LOW, MEDIUM and HIGH risk, stage transitions, warnings, attempts at fixing the problem, and reporting. The sample is thus ideal for showing system behavior while the Alibaba and Google traces provided benchmark evidence for the prototype.

3.5 Instrumentation and Development Tools

The principal research instrument is the implemented software prototype. The prototype consists of Python modules for loading data, extracting features, training models, comparing classifiers, monitoring live logs, adapting external process output, generating reports, rendering the dashboard, and defining the Airflow DAG.

Python was used because it provides mature libraries for data manipulation, machine learning,

dashboard development, and process automation. The system requires the following libraries: XGBoost, scikit-learn, pandas, NumPy, Matplotlib, Seaborn, Streamlit and psutil. Trained models have been saved in models /folder. All reproducible outputs as well as comparison charts have been saved in outputs/ folder.

Table 3.3: Implementation modules used as research instruments

Module	Main responsibility	Defence relevance
data_loader.py	Loads HDFS, Google, Alibaba, and Google 2019 records.	Shows how different data sources enter the system.
feature_extractor.py	Converts raw records and streaming logs into numeric features.	Explains model inputs and early-warning evidence.
model.py	Trains XGBoost and stores evaluation artifacts.	Supports the model-development objective.
model_comparison.py	Compares baseline and candidate classifiers.	Supports the algorithm comparison objective.
pipeline_monitor.py	Wraps streaming prediction and risk calibration.	Connects model output to LOW, MEDIUM, and HIGH risk.
predictor_plugin.py	Provides the plugin-facing API and report payload.	Represents the project as an attachable monitoring plugin.
live_pipeline_runner.py	Runs external commands and streams logs into the plugin.	Demonstrates live attachment to a pipeline process.
etl_pipeline_demo.py	Implements the extract, validate, transform, load, and report demo pipeline.	Makes the defence demo feel like a recognizable pipeline.
dashboard_app.py	Renders the Streamlit dashboard.	Provides the visual representation required for defence.
dags/pipeline_failure_monitor_demo.py	Defines the Airflow DAG.	Provides orchestration evidence.

The core artifact produced by the research was the software prototype developed, which produces the model artifacts, outputs, monitoring outputs, dashboard and reports considered by the study. Python was the main programming language used, due to the wealth of data science and machine-learning libraries available, many of which are quite mature (e.g. Pandas, NumPy, scikit-learn and XGBoost). It was found that scikit-learn, in particular, was very helpful for the creation of training sets, training of comparative models and calculation of metrics (Pedregosa et al., 2011).

The evaluation tools were chosen in line with the research aims. The F1-score was included because it combines the strengths of precision and recall and is a more appropriate measure than accuracy when failures are infrequent (scikit-learn developers, n.d.-b). Stratified and grouped validation was critical because related snapshots, captured from the same job, cannot be blindly

treated as independent training/test data points (scikit-learn developers, n.d.-a).

The dashboard framework utilized was Streamlit, enabling a browser interface to display data controls, charts, logs, tables, and downloadable output files that were created using only a Python language (Streamlit, n.d.). Apache Airflow was chosen to demonstrate orchestration because it allowed a pipeline to be conceptualized as a set of tasks in a Directed Acyclic Graph (DAG) structure, with configurable task behavior (Apache Software Foundation, n.d.). Together, Streamlit and Apache Airflow enabled the transition from an experiment in machine-learning to a deployable monitoring workflow.

3.6 Data Collection and Preprocessing

The project employs two sources of secondary public traces and runtime ETL evidence. The public traces are read by code-based loaders instead of directly edited text files. This is essential for the iterative, reproducible, and verifiable reprocessing of raw data without modification.

The HDFS loader breaks log lines into their date, time, process id, severity level, component, and content fields. The Alibaba loader reads members from compressed archives, merging job-level and task-level evidence. The Google 2019 loader reads locally stored rejoined trace export and maps workload fields into numerical values. The ETL demonstration yields structured logs from live pipeline stages.

Labels are also part of preprocessing. Success and failure labels are derived for supervised learning and evaluation. Live inputs for prediction cannot contain terminal outcome fields. Otherwise, the model would have had access to the answer (label) post-completion of the job; it would only seem correct, leading to a leak in accuracy.

Data was collected using reproducible loaders and on-the-fly generated runtime evidence. These two data types differed because the public traces were processed as stored datasets, while live logs and the ETL demo were data from a running process. Although both data sources were valuable, they necessitated different preprocessing strategies.

Preprocessing with offline datasets was composed of identifying semantics of schema, inputting values for unknown entries, generating labels, and field translation or normalization when necessary. Preprocessing of live streams included identify schema of raw texts (e.g., timestamp identification, severity identification), identify jobID, extract component information and

normalize it. Hence, the workload-trace preprocessing is separated from the streaming log preprocessing in this work, rather than treating all sources identically.

The avoidance of label leaks was an important constraint during preprocessing. Final outcome fields that directly indicated success or failure were used to form the training labels, but not as predictive inputs, thus protecting against inflated accuracy values unachievable in real-time monitoring scenarios. The features used in prediction ought to consist of pre- or during-run indications of trouble, not the solution at the end of the process.

This preprocessing step also helped increase reproducibility. The loaders, feature generators, and demonstration scripts represent the mapping between an individual observation and a modeling record, thereby improving the credibility of the research beyond a manual workflow, by enabling replication of and verification of the process.

3.6.1 HDFS-Compatible Log Preparation

HDFS-compatible preparation grouped the lines with the aid of the stable block/job id. The adapter converted external messages into that format so that feature extraction from the stream could use a known schema, where each individual message preserved its timestamp, level, component, message, stage and job identity.

The HDFS-compatible preparation has been organized around grouping evidence of execution. Log lines are keyed by stable identifier to permit the feature extractor to summarize over a window (a job or a single observation) and across time. The reason for grouping by stable ID is because a single log entry might be meaningless, but a stream of consecutive errors and retries signals a propagating fault.

The live ETL adapter works similarly. It took incoming pipeline messages and converted them to the stable structure and its included pieces of information (timestamp, severity, component, stage, message and job identity). The monitoring logic would then increase its features, instead of rewriting the whole ETL pipeline as a machine-learning script.

3.6.2 Alibaba PAI Preparation

The Alibaba preparation was oriented toward workload and task summarizations. The training data generated such metrics as number of tasks, number of failing tasks, time for scheduling delays, request of resources, number of finished task status; these are adapted to prediction at

workload level, and not for natural language log anomaly detection.

The Alibaba preparation work was performed on workload, and not on natural language log files. The task and job data were summarized into specific fields, for instance task quantity, unachieved tasks quantity, scheduling delay quantity, resource request quantity, final status; these are adapted to a training table format because they directly characterize the work structure and its accomplishment at cluster level.

Size of original data trace was considered. Huge, compressed archives were not committed into the deployable repository. Instead, model artifacts and comparative summaries were kept for dashboard and training evidence: the dashboard was small and the training result was able to be clarified in the report.

3.6.3 Google 2019 Preparation

Google 2019 preparation uses a rejoined local export of the Borg trace as published by the Borg trace schema. This Borg trace was used as a distinct source because its feature schema was different from HDFS and Alibaba, thus avoiding the inaccurate assertion that one unified feature schema was trained over sources with disparate feature sets.

The Borg trace was treated as distinct source in the Google 2019 preparation as its schema was different than both the HDFS-compliant and Alibaba sources, so available fields from the Borg trace were mapped to numeric features, with the Borg trace label retained for the failure prediction.

This prevented the logical fallacy that one model had been trained over incomparable feature schema. The Borg sample also provided a useful test bed of interpretation. Although Google data performed less impressively than the controlled ETL testbed, allowing the report to come to a realistic, though conservative, conclusion. This suggests the model used can best be thought of as a proof-of-concept and benchmark implementation rather than a production prediction model.

3.6.4 ETL Demonstration Preparation

The ETL demonstration uses data/sample_orders.csv as a small deployable input file. The running ETL stage emits logs at the level of individual ETL stages, where data is pulled, the fields are validated, valid rows are transformed and loaded into an output store and a summary report is written. Due to the small, reproducible nature of this file, it is a good input for Render

deployment and defense demonstration.

The ETL demonstration was designed to be small and repeatable and to be harmless to run during a defense session. A tiny sample orders file is processed, well-known ETL stages are exercised, and logs are emitted for the usual operation of the system as well as warnings, recoverees, and potentially dangerous conditions. This gives the evaluator something tangible to observe as opposed to simply a raw file.

By giving a controlled process, the ETL demonstration is transparent. Intentionally selectable scenarios ensure that various system states can be seen through the dashboard, which helps the report make a distinction between the demonstrative function of the model evaluation evidence and the claims that might otherwise be inferred about production systems.

3.7 Feature Engineering

Feature engineering is the bridge between raw operational evidence and machine learning. The implementation does not take raw log data to directly feed into XGBoost, rather, it aggregates each job or time window of observation into numeric features that represent severity, frequency, recency, workload structure, resource information and repeated failure information.

Feature extraction from HDFS-compatible log records takes the form of the total number of lines in a log, counts of each severity level, the rate of warnings, the rate of errors, any fatal, the rate of retries, certain keywords (like exception, fail, stop, timeout), unique components and process IDs, a maximum burst of errors, and temporal features. These features lend themselves well to the early warning detection framework as they can be re-computed as log records come in.

Workload feature extraction from the Alibaba and Google workload traces are tailored to capture aspects of workload structure: the number of tasks, number of failed tasks, scheduling latency, CPU request, memory request, GPU request, priority field, sample of runtime usage, labels derived from the status of the task. These features are distinct from HDFS-compatible features.

Table 3.4: Feature engineering categories

Feature category	Examples	Reason for inclusion
Severity profile	INFO, WARN, ERROR, FATAL, DEBUG counts and rates	Shows whether the execution stream is becoming operationally unhealthy.
Failure keywords	exception, failed, timeout, retry, stopped, invalid	Captures text patterns commonly associated with failure or recovery attempts.

Feature category	Examples	Reason for inclusion
Burst behavior	maximum consecutive error count	Distinguishes isolated warnings from clustered degradation.
Execution diversity	unique components and process identifiers	Shows whether many components are involved in the incident.
Workload structure	task count, failed task count, scheduling delay	Represents job-level complexity and unsuccessful work.
Resource context	CPU, memory, disk, GPU request, usage snapshot	Captures operational pressure that can contribute to failure risk.
Temporal evidence	elapsed time and early-warning snapshot fraction	Supports prediction before terminal failure.

Feature engineering is treated as the core research question because the resulting models depend on how well the observed evidence was prepared for them. Natural language log messages can be problematic inputs for a tabular machine-learning model and thus, the system converts logs and workload traces to numeric signals. Signals describe severity, frequency of messages or task failure, timing between messages or task requests, resource information and workload information.

For the log based evidence the most valuable features are those that can be detected while a process is running: error counts, warnings counts, ratio of aberrant log messages, repetitions of keywords of exception messages, detection of timeout and retry messages or detection of error burst are relevant because these symptoms can be observed before the job actually fails.

For the workload trace-based evidence, the relevant features are different. The counts of tasks, the counts of failed tasks, scheduler delays, resources requests, snapshot of usage and priority field explain how a job behaves with the cluster. These features are in line with the studies that indicate that the behavior of a job influences the result of the task. Specifically, resource usage, scheduler behavior and task level instability can determine the result of the task (Yuan et al., 2012). Cloud-job failure prediction research also supports the value of relating job behavior to machine-learning features (Jassas & Mahmoud, 2022).

The implementation deliberately kept feature sets source specific. A feature such as failed task count is meaningful in a workload trace, while a feature such as maximum consecutive ERROR lines is meaningful in a log stream. Keeping these feature sets separate improved methodological validity because it allowed each model to learn from the evidence type that exists in its data source.

3.8 Model Development and Validation

The model development process takes the form of iterative steps: load labelled records, feature extraction, data splitting, candidate model training, held-out predictions, charting, and storage of the final chosen model artifact. While the chosen family of models deployed is XGBoost, comparison artifacts are kept, allowing for reporting how well it did relative to other classifiers.

Separate model artifacts are kept for HDFS compatible/ETL demonstration, Alibaba, Google 2019, and legacy/default HDFS compatible model. This is an important methodological choice. A single combined model would need to support a universal cross-source feature schema, and a cross-domain validation; neither of which is explicitly made here.

Held-out model metrics, along with comparison metrics are used for validation. In the case of repeated snapshots being drawn from one process run, a grouping mechanism is put in place to avoid leakage by not treating correlated snapshots as completely independent evidence both in testing and in training. Among other metrics listed in the final model artifact and model-comparisons.md artifact file, performance is charted using confusion matrix counts, precision, recall, F1-score, ROC-AUC, PR-AUC, MCC, specificity, log loss and inference latency.

For reproducibility, the implemented XGBoost configuration used 300 estimators, a maximum tree depth of 5, a learning rate of 0.08, subsampling of 0.8, column sampling by tree of 0.8, and random_state set to 42. The train-test split reserved 25% of the available records for held-out testing. Where process or group identifiers were available, the implementation used a group-held-out split so that related snapshots from the same job were not divided across training and testing partitions.

The validation code also used five-fold cross-validation. StratifiedGroupKFold was applied when repeated snapshots or group identifiers made grouped validation appropriate; otherwise, stratified row-based folds were used. This explicit validation strategy addresses leakage risk and makes the model-development process more reproducible from the stored source code and artifacts (scikit-learn developers, n.d.-a).

Table 3.5: Stored model artifacts

Artifact	Role	Use in defence
xgboost_pipeline_monitor.pkl	Legacy/default HDFS-compatible model	Kept as an earlier artifact and fallback path.
xgboost_pipeline_monitor_etl.pkl	ETL/HDFS-compatible live demonstration	Used by the dashboard for the live ETL

Artifact	Role	Use in defence
	model	demo.
xgboost_pipeline_monitor_alibaba.pkl	Alibaba benchmark model	Shows source-specific benchmark training.
xgboost_pipeline_monitor_google2019.pkl	Google 2019 benchmark model	Shows second production-trace benchmark training.

The model-development phase included a baseline assumption that the chosen model must perform better than a majority-class classifier and that its performance should be evaluated against alternative machine-learning algorithms because failure-prediction can be biased due to class imbalance. High accuracy may be achieved through a majority-class classifier predicting success for all jobs, yet this is an unhelpful model, as it fails to recognize when jobs are failing. Consequently, the validation stage was focused on failure-class performance, with precision, recall and F1-score used to assess how successfully the model correctly identifies true failures and how accurate its identification of failures is, respectively, with F1-score being a balanced representation. The inclusion of PR-AUC was also due to its value for imbalanced classification problems where the positive class is critical. Other metrics ROC-AUC, specificity, MCC, log loss and inference latency were all preserved due to answering diverse questions regarding the classification performance: ranking quality, successful job prediction not being flagged as failure, overall balanced performance metric, the quality of probability prediction and the speed of prediction for use in a live monitoring application, respectively.

Model artifacts were retained in validation because the process was reproducible: rather than relying upon screenshots for the experiment's results, trained model files, metrics, comparison charts and tables were kept. This provides more useful evidence than purely decorative presentation images and distinguishes between a tangible model pipeline and a screen. The validation intended to illustrate operational usefulness rather than a large numerical score. Running at a large scale a false negative means that we get a failure that runs unnoticed; a false positive means that we get an unwanted shutdown. Because of this the confusion matrix seemed to be a reasonable metric, where the elements were: false negatives-a successful job that we say is a failure, false positives-a successful job that we say is a failure, true positives-a failure we detect, and true negatives-a failed job that we say is a failure.

A majority-class baseline was used due to class imbalance in the typical failure prediction dataset. If most jobs are not failing, any classifier would obtain a high score simply by always

predicting success, which is not helpful for this application. As such, the purpose was to determine if any significant failure warning patterns had been learnt rather than only the majority distribution, meaning a majority-class baseline provided a useful comparative benchmark.

The validation outcomes were saved as project artifacts to be referred back to in writing and defence, and for potential future enhancement. The saved artifacts include charts, tables of comparisons, the numerical outputs and model files, proving the validity of the reported figures as they can be linked to reproducible files generated during implementation rather than arbitrary figures inserted at writing time. The model-validation was not generalized since the ETL/HDFS-compatible model, Alibaba model, and Google 2019 model represent specific inputs which have different variables and operational relevance. This decision maintains the honesty of the assessment because a successful result within the controlled ETL model provides confidence in a live demonstration whereas a poor performance result for the other models highlight the need for greater input data and refinement of hyper-parameters.

3.9 System Architecture and Implementation

The architecture of the system revolves around the concept of a dynamically attachable monitor. A pipeline process could be run and monitored by the monitoring layer, collecting logs, taking samples of runtime statistics, updating features, running a prediction model, classifying risk, and recording the monitor session. As a result, this implementation not only covers the software workflow, but also the decision support workflow.

The system's architecture based on attaching to a live system is distinguished from a static notebook system by the presence of a subprocess runner, a log adapter, a streaming extractor, a predictor plugin, and a dashboard state layer. These provide an implementation which resembles a true monitoring plugin more closely in that the predictor and pipeline are different objects.

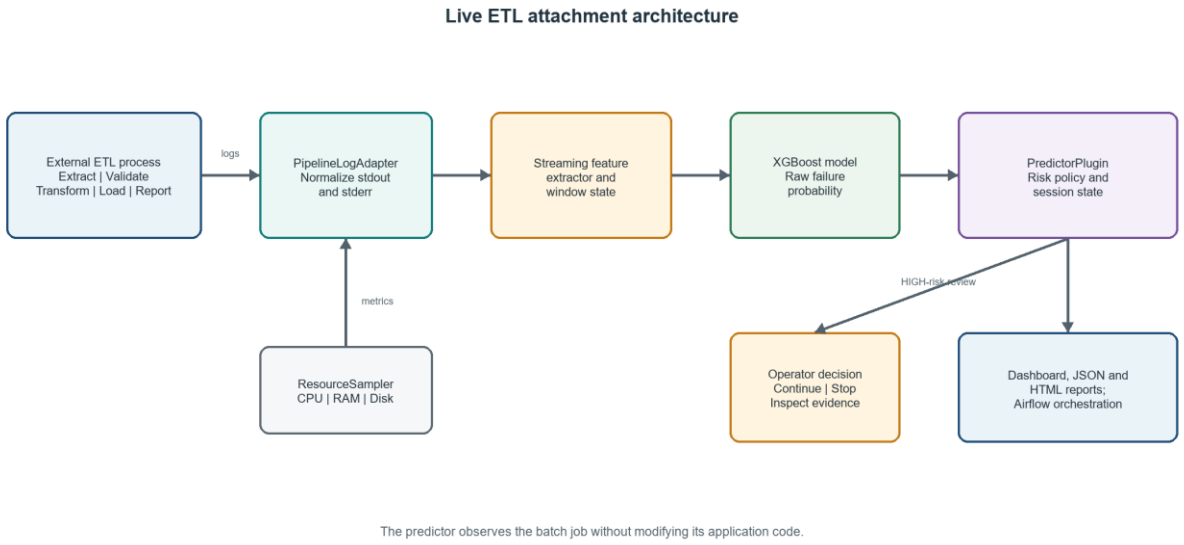

Figure 3.2: Live ETL attachment architecture

It follows a separation of concerns pattern. ETL service produces the raw events, runner picks them up, adapter transforms them, feature extractor summarizes the events, the model scores for risk, plugin writes the alerts and reports, and dashboard displays the session. The benefit of this is it's easier to understand the system as each piece of the system has its responsibilities.

The attachable monitoring idea is significant because many pipelines already exist and there is need to introduce the prediction service. Re-writing every pipeline would cause much friction for usage. The live runner therefore shows the method by which a predictor could run alongside an external command and observe its standard output, error, and performance without having to transform the external command into a notebook.

This system is also designed with auditability in mind. A prediction made without its logs, stages, and report is one a operator would likely be hesitant to trust. Thus, this system persists risk counts, high-risk items, recent logs, summary of resources used, summary of stages executed, and report data for retrospective analysis.

3.9.1 External ETL Batch Process

It is a process called "ETL" for Extract, Validate, Transform, Load, report. The process read order data sample, validates record fields, transformed admitted records, writes filtered output, and emits trace. The demonstration can walk through different risks using normal, warning,

recovering, risky, and mixed scenario controls.

The external ETL batch was built around a well-known process for data engineers: extract, validate, transform, load, report. Each phase sends progress messages allowing the monitor layer to trace normal execution or abnormal execution, thus make the demonstration more believable than a random event generator as it follows an observable structure.

The scenario controls give repeatability; for example, normal scenario to demonstrate stable execution, or warning, recovering, risky and mixed scenario to demonstrate various alert paths. These controls would help examiner understand the intervention mechanism better.

3.9.2 Live Attachment Runner

The live runner executes the command, collects stdout and stderr, normalizes each line and then sends the modified record to the predictor plugin. It additionally captures the current CPU, memory and disk usage information, where psutil can be found and sends the collected data too. Thus, the dashboard can display not only log messages but also provides a context of the execution.

The live attachment runner is the component that links the execution to the monitoring. It executes the command, captures its input streams, keeps some previous records and then sends normalized records to the predictor plugin. The existence of this runner illustrates how the prediction system can be provided with evidence about a process during its execution, as it is more similar to actual monitoring than training or testing only on an isolated file of CSV records.

The execution context is provided when possible. The CPU, memory and disk readings allow the dashboard to understand if a message that is considered as an anomaly also occurs when system resources are stressed. However, those samples are not meant to provide a full infrastructure observability platform but rather to increase the usability of the prototype, adding an extra contextual layer.

3.9.3 Predictor Plugin

The predictor plugin acts as the re-usable wrapper for the monitor. It ingests lines, saves alerts, recognizes HIGH risk, sets up the review state, produces report payloads, and writes reports both as HTML and JSON. It exposes a plugin style API to the project and doesn't allow the prediction code to be embedded within one script.

The predictor plugin is where the monitoring process is made re-usable. It avoids dispersing prediction code throughout the dashboard, ETL script, and report generator by taking the standardized records and using them to update the monitor, save the alerts, maintain the high-risk status and produces report payloads. It establishes a better software boundary for the project.

The plugin design also helps facilitate the policy driven behavior of the application. The same prediction engine is now capable of operating in Advisory, Review or Auto-Stop mode depending on the environment in which the monitoring tool is being deployed. It is not always advisable for every HIGH-risk prediction to kill a process and, some environments may need approval, other times the cost of processing is too high to justify continued execution.

3.9.4 Streamlit Dashboard

The dashboard has controls, stage boards, logs, samples of the resources, risk cards, model comparison charts, system maps, and report downloads sections. It is the visual user facing aspect of defence. It needs to be framed as the operator dashboard rather than the machine learning model itself.

The Streamlit dashboard is the operator and examiner-facing layer of the prototype. It presents controls, stage indicators, logs, metrics, risk cards, charts, reports and system maps together in one place. This helps with the aim of displaying the prediction workflow, not just printing a model output to the terminal.

The dashboard has been constructed to make the evidence visible; a user is able to display the run scenario, the current stage, the current risk level, recent alerts and outputs from reports. This is critical as an operator's belief in a failure prediction tool relies on their ability to see and interpret why a run is being presented as risky.

3.9.5 Airflow DAG

The Airflow DAG presents orchestration evidence. It describes the following tasks: prepare_run, run_pipeline_with_predictor, evaluate_intervention, publish_report. It illustrates where the predictor could reside within a task-based workflow, yet it is not depicted within the Render Streamlit service.

This DAG was provided as evidence of how to "put a monitoring within an orchestrated workflow." It provides explicit clarity around the process of orchestrating the components within

the system and the process of building out a workflow in Airflow. That is, that there is a process of “prepare a run,” then there is the “execution” itself (with the predictor included “within” it), then the “evaluate” step, and finally the “publish” step. This gives strong evidence of the overall workflow and support for the idea that the “prototype” can be framed as a “workflow.”

The DAG was also structured as narrowly scoped evidence. The DAG is presented as evidence of “orchestration design and local workflow integration” rather than “Render web service design.” Doing so avoids overpromising and over-claiming.

3.9.6 Render Deployment Preparation

Deploying to Render is set up for the Streamlit dashboard. The repository contains: render. yaml, ..python-version, requirements.txt, trained models, small samples of the data, and graphs of the output. The raw larger traces, and local report drafts, are not included.

The work for Render deployment was tailored to the web application of the Streamlit dashboard, as this was the part of the workflow that lends itself best to a light, hosted demonstration. Render needs the specified render.yaml and ..python-version to set up and the requirements.txt to properly download dependencies and to get the dashboard running with the appropriate host and port arguments.

The larger raw traces, local report drafts, and generated outputs were omitted because including these larger, often generated, files will cause large repositories that do not contribute significantly to a demonstration of the dashboard's potential, and were intended to remain locally for heavy processing.

3.10 Risk Classification and Intervention Logic

The system uses two thresholds from config.py to classify risk. A probability greater or equal than 0.70 is classified as HIGH, from 0.40 and up until 0.70 as MEDIUM, and anything below 0.40 as LOW. This provides a simple and explainable risk category scale for both the dashboard and the report output.

The system also ensures to retain raw model probability when demonstration calibration is applied. In pipelinemonitor.py the calibrated outputs are stored with raw_model_failure_probability to allow a varied percentage integer during simulation while retaining all model evidence. Such distinction is essential to avoid a synthetic presentation value

from becoming an evaluation metric in the report.

The HIGH-risk category does not always trigger automatic pipeline stoppage, rather supports the plugin to comply with a policy. For advisory and review modes the HIGH risk triggers a request for inspection and a decision made by the user to proceed or stop. In auto-stop mode, the system automatically stops the pipeline upon configuration, as is common practice and often required prior to termination of a business process in production systems.

The categories HIGH, MEDIUM and LOW translate model probability to a simple operator-friendly language understandable by anyone regardless of their technical background. It transforms model probability to a format usable for any component of the system from the dashboard to the report and, decision support.

The threshold values chosen are intended to be obvious in their definition as to ease the explanations in case of defense, with a HIGH threshold at 0.70 and MEDIUM at 0.40. The report acknowledges that actual production thresholds are expected to be based on estimations of cost associated with potential damage of missed failure and false alarms.

The choice of intervention also relies on the rejection of an unsafe decision where the model automatically makes the final operational choice. The review mode permits a flagging as HIGH while decision remains with the user.

The need for a risk classification layer instead of using model probabilities directly comes from the fact that probabilities themselves can be hard to understand intuitively for any operator of the system; by transforming these to three discrete categories it allows the system to develop its own common operating language that anyone can interpret in real-time, for the report as well as the intervention decision, allowing to visualize how a model's output becomes a warning for an examiner. The classification scheme also supports reviewability: these values are hardcoded and another user or researcher can easily change them to see the new outcome.

These thresholds should not be fixed and are not in production but are based on estimations of business costs for missed failures and false positives. The intervention distinguishes between prediction and intervention: HIGH risk determines a warning and the decision whether the system should be paused, have a human review it, or completely stopped depends on a defined policy. This allows each organization to define risk based on its own needs as a financial

company for instance, may choose to halt the pipeline entirely, whereas an experimental pipeline would probably be placed in review mode.

3.11 Method of Data Analysis

The analysis draws evidence from both model performance metrics and system-functionality data. Model-performance data is provided by the saved metrics files and the comparison files. System-functionality data is drawn from executable code paths – the live runner, the ETL process, plugin, dashboard, report generator, and Airflow DAG.

Metrics such as accuracy, while reported, can be a poor measure of whether the system detects failures effectively. Because jobs succeed more frequently than they fail, accuracy can be very high on an imbalanced system even if failures are not predicted. Precision, recall, F1-score, PR-AUC, and confusion-matrix cell counts offer better insight into how the failure class is behaving.

The analysis was done at two levels: model level (interpreting held-out metrics and comparison outputs) and system level (examining behavior by examining the dashboard, runner, plugin, report generator, and Airflow DAG). Neither analysis level is sufficient on its own. It was necessary to show both that the metrics were predictive and that the entire monitoring workflow functioned.

A more refined level of detail separates controlled data (ETL/HDFS compatible output is expected to be stronger than either Alibaba or Google, where data is both source-specific and imbalanced) from production style data, and this level of analysis is maintained throughout chapter four. Evidence is the result, not decoration. Metrics, charts, reports and dashboard snapshots are all directly linked to executable code paths, which provides traceability and greater confidence in the results.

3.12 Validity, Reliability and Reproducibility

Keeping source-specific schemas allowed for maintenance of validity and for avoidance of a false false-merged-model claim. Reliability was maintained by the modular structure of the code. Reproducibility was provided by saving model artifacts, a small usable sample of test data, saving comparison results, and recording the run command. The implementation has some known bounds.

The ETL demonstration is controlled. The Alibaba and Google sample data is limited. The

dashboard could make use of presentation calibration for demonstrations. The Airflow code was included as proof of concept for orchestration but is not started within the Render web service. The boundaries of the system have been noted openly in order to make the project defensible on an academic level. Internal validity was addressed by not introducing any data leakage; by preserving source-specific schemas; and by comparing the selected model against a few alternatives. These aspects reduce the risk that the observed results are due to artificial shortcuts in preprocessing or a not examined model choice.

The limitations in this project were openly presented where the evidence had to be treated as bound or demonstrative. Reliability was addressed through the implementation of being broken into modules for various tasks. These included data loading, feature creation, model training, comparison, live running, prediction, reporting, and dashboard rendering. This was implemented so as to make the system easy to test and alter without needing to change the system as a whole. Reproducibility was addressed by saving all model artifacts, metrics, graphs, sample data and configuration files. Future reviewers of the project will be able to study the files used within the dashboard and compare it against the descriptions contained within the report. Full trace retraining, while possible, has not been demonstrated within the deployable demonstration, as a bounded basis for experiments was achieved with the use of saved artifacts.

Construct validity was supported through features being closely tied to recognizable operating warning signs. The models used severity counts, retry indicators, exception keywords, timing information, resource information, and count of failed tasks, which are all interpretable features. Though not automatically proving the cause, these variables were more readily defendable than some completely abstract or obscure features would have been. External validity is still limited since the live ETL demo is controlled and the public trace experiment is bound. Consequently, no claims were made that the demonstrated model would be generalizable to any other log file or larger traces without retraining.

The actual claim is that the architecture presented in this project offers a practicable failure-prediction system and monitor that could be generalized to different traces and logfiles. Reliability was further supported by ensuring the difference between the training artefacts and presentation artefacts is preserved. Whereas the dashboard could utilize rounded-or-calibrated values for ease of viewing in a presentation, the raw probability outputs from the models as well

as the saved evaluation metrics remain readily available for academic review. The differentiation of the values allows the presentational layer of the tool to be readily distinguished from the presented data.evidence.

3.13 Ethical Considerations

Our project employs public traces and synthetic demonstration data, not any data from human subjects. Real-world use would require that logs be reviewed to mask any personally identifiable or sensitive business data. The system prototype deliberately refrains from overselling readiness and avoids blurring the lines between model metrics and demonstration effects. Because we do not use any human participant data ourselves, we may sidestep some ethical challenges for the system's prototype.

However, some logs may in any event identify people or include sensitive data. In a deployed application, the service would need procedures to mask credentials, manage report and log views and access controls, and mask personally identifiable information. We side-step such concerns by working with the readily available data. We also frame model limits as ethical concerns: for example, the prediction results affect the pipeline's stop, hold or escalation procedures, and therefore we refuse to say our model has achieved production readiness without broader and deeper studies of operational settings. Instead, we position this prototype as one element of support to assist human decision making, but not substitute for it..

3.14 Summary

This chapter describes research design, the data sets, choices about sampling, tools, pre-processing, feature engineering, model development, system architecture, risk policy, validation, reproducibility, and ethical issues. Chapter 3 describes the system implementation and stored evaluation evidence.

This methodology connects supervised learning, prototypes, source-specific models, live monitoring of the ETL, visualization through dashboards, generation of reports, and orchestration evidence. This chapter details why samples bounded are used, why the public traces aren't consolidated in a single model, and why demonstration behavior is separated from the measured evaluation evidence in the dashboard, leading into the results of chapter 4.

CHAPTER FOUR

DATA PRESENTATION AND ANALYSIS

4.1 Preamble

This chapter shows the developed system and the logged results generated by the project. It is only concerned with existing artifacts and existing behavior, and makes no claims regarding external production deployment, complete tracing retrainings, or unproven Airflow running behavior other than the developed DAG.

This chapter will discuss the evidence in a bifurcated way. It discusses the implementation evidence (which verifies that the developed software operates as a monitoring system), separately from model performance evidence (which addresses how the trained classifier performs on available evidence). The separation between these two pieces of evidence is important, as the presented dashboard may appear complete while the model is poorly performing, while a model could test well, yet be poorly integrated into a working system.

It also discusses the limitations of each type of evidence. The staged ETL experiment, while useful for testing the monitoring pipeline, doesn't provide much variety; the Alibaba and Google samples, on the other hand, test the methodology over samples with varying characteristics. For this reason, the results will be discussed as prototype evidence and benchmark evidence rather than an attempt to prove enterprise-level production validity.

4.2 Implemented System Output

There are three paths demonstrated in the final prototype: the synthetic simulation path (for controlled HDFS-style events), the monitored ETL path (for executing the ETL process of extract, validate, transform, load and report), and the attached command path (where a given command is run by LivePipelineRunner, and its output streamed through the predictor plugin).

These paths are presented to the user via the dashboard interface, which shows the user a possible way to monitor a data pipeline while it is being executed. It includes information about the current stage, recent log messages, count of risks, current risk level, risk descriptions, sample resources, intervention status, comparison graphs and reports; which directly mitigates the lack

of any information than simply terminal output.

Table 4.1: Implemented demonstration modes

Mode	Code path	Purpose	Evidence
Synthetic simulation	simulation_engine.py	Controlled event stream for dashboard behavior.	Scenario selection, stage board, varied risk states.
Monitored ETL	etl_pipeline_demo.py	Realistic CSV pipeline with extract, validate, transform, load, and report stages.	Live ETL dashboard and report output.
Attached command	live_pipeline_runner.py	Plugin-style attachment to an external command.	Raw and normalized logs; resource samples.
Airflow orchestration	dags/pipeline_failure_monitor_demo.py	Workflow evidence for ordered tasks and intervention decision.	DAG tasks and configurable parameters.

The system output described above proves that the work did not remain a static prediction. User is able to initiate a run, pick a scenario, watch the stage progress, analyze the logs, watch risk counts vary, search through HIGH-risk messages, and generate a report. These indicate the predictor is part of an executable workflow like that of an operator monitoring a batch job.

The three demonstration paths were created with different evaluation goals. Synthetic simulation demonstrates that the dashboard and the risks appropriately react to input data streams. Monitoring of an ETL path demonstrates that it can read and generate logs/reports from an already running process using a plugin. And the command path attached proves that monitoring concept is portable to a foreign command execution. Together they demonstrate that predictor is not limited to one script.

Defense finds this particularly useful since cause and effect can be viewed. When warnings/errors increase, feature extractor changes the evidence, risk increases, and dashboard updates the user to show the changed condition. This is a much more intuitive explanation to an isolated metrics table.

4.3 Model Evaluation Results

4.3.1 Failure Pattern and Feature-Importance Results

Objective 1 required the study to identify and characterize common failure patterns in data pipeline batch jobs. The saved feature-importance chart from the ETL/HDFS-compatible XGBoost model shows that the strongest warning signals were error_rate, info_count, error_count, max_error_burst, warn_count, and warn_rate. These features indicate that failure

risk in the demonstration model was driven mainly by the concentration of error messages, the volume of normal and abnormal log activity, repeated error bursts, and warning density.

The result is operationally meaningful because the strongest features are observable while a pipeline is running. A high error rate or clustered burst of errors can alert the system before the final job outcome is known, while warning counts and warning rates can indicate degradation before failure. This closes the loop between the first objective, the feature-engineering method in Chapter Three, and the evaluation evidence presented in this chapter.

Table 4.2: Feature-importance and failure-pattern evidence from the ETL/HDFS-compatible model

Top signal/pattern	Evidence from saved feature-importance chart	Operational interpretation
Error density	error_rate and error_count ranked among the strongest predictors.	Frequent errors indicate unstable execution and possible terminal failure.
Burst behavior	max_error_burst appeared as a major feature.	Repeated consecutive errors are more serious than isolated messages.
Warning activity	warn_count and warn_rate were visible among the leading features.	Warning accumulation can show degradation before a hard failure occurs.
Execution volume	info_count and total/log-line indicators contributed to the model.	The amount of execution evidence helps distinguish normal progress from abnormal runs.
Failure keywords and retries	Keyword and retry features contributed at lower levels.	Textual failure hints support prediction when combined with severity and burst features.

The primary stored ETL/HDFS-compatible metrics achieve an accuracy of 0.9250, precision 0.8421, recall 1.0000, F1-score 0.9143, ROC-AUC 0.9883, and PR-AUC 0.9704. These results are a validation of the controlled live demo, but do not guarantee the same performance across all production pipelines; they do require further testing.

The Alibaba and Google 2019 benchmark shows less promising results. This is likely because the production-like workload traces are inherently more disparate and imbalanced. Because of this, the benchmark is cited as one supporting its effectiveness rather than proof of its generalization across all live ETL.

Table 4.3: Held-out XGBoost evaluation results preserved from project artifacts

Metric	ETL/HDFS-compatible model	Alibaba PAI sample	Google 2019 sample
Accuracy	0.9250	0.8768	0.6647
Precision	0.8421	0.2244	0.3258
Recall	1.0000	0.5147	0.5733
F1-score	0.9143	0.3125	0.4155
ROC-AUC	0.9883	0.7267	0.6709
PR-AUC	0.9704	0.3415	0.3057
MCC	0.8584	0.2830	0.2207
Specificity	0.8750	0.8976	0.6887
Log loss	0.1222	0.4328	0.7793
Confusion matrix	TP 16; TN 21; FP 3; FN 0	TP 35; TN 1,061; FP 121; FN 33	TP 301; TN 1,378; FP 623; FN 224

Table 4.4: Classifier comparison for the ETL/HDFS-compatible experiment

Algorithm	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.992	0.963	0.963	0.963	0.9977
HistGradientBoosting	0.992	1.000	0.9259	0.9615	0.9887
XGBoost	0.988	1.000	0.8889	0.9412	0.9894
Random Forest	0.980	0.9231	0.8889	0.9057	0.9475
Decision Tree	0.976	0.8889	0.8889	0.8889	0.9437
Majority Baseline	0.892	0.000	0.000	0.000	0.5000

Table 4.5: Classifier comparison for bounded Alibaba and Google benchmark samples

Dataset	Best F1 model	Best F1	XGBoost F1	Interpretation
Alibaba PAI sample	HistGradientBoosting	0.4324	0.3238	The production-trace sample is imbalanced and harder than the controlled ETL demonstration.
Google 2019 sample	Random Forest	0.4409	0.3234	The random forest comparison produced stronger F1 than XGBoost on this bounded sample.

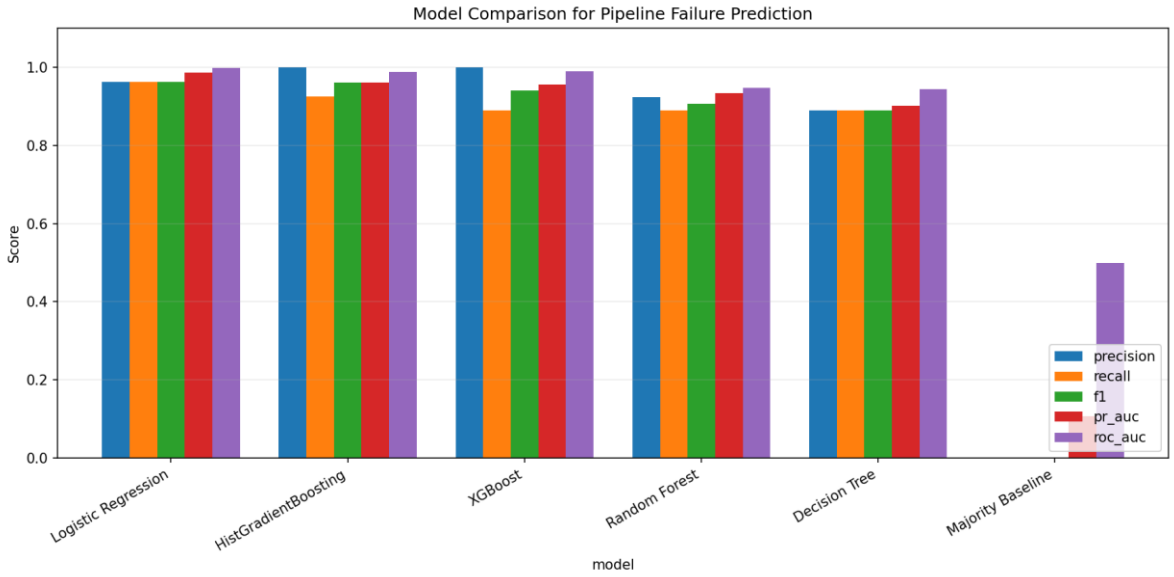

Figure 4.1: Model comparison chart for ETL/HDFS-compatible experiment

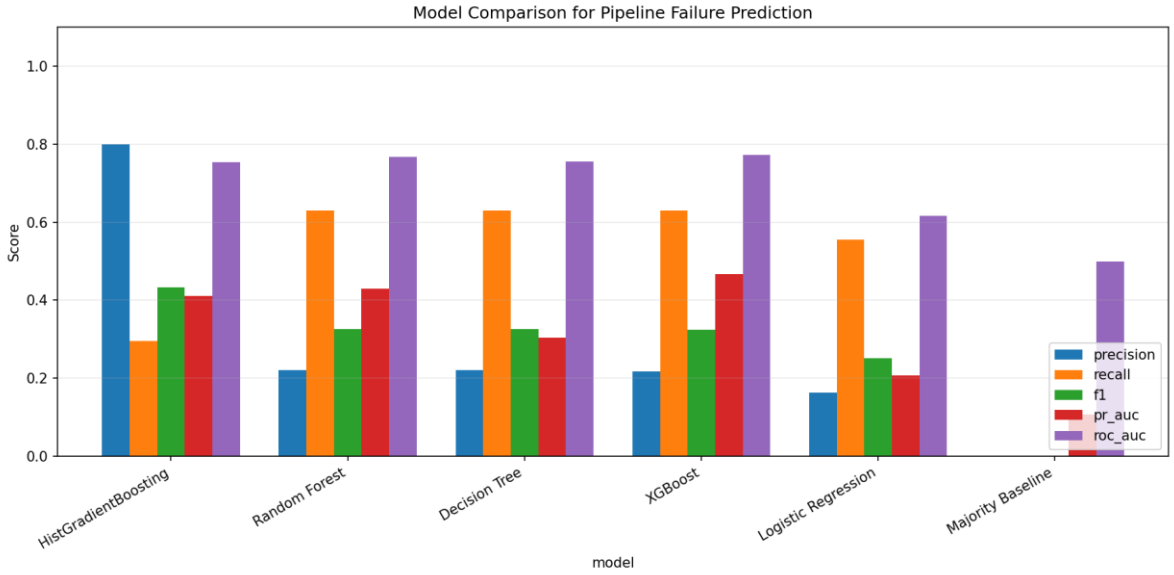

Figure 4.2: Model comparison chart for bounded Alibaba PAI sample

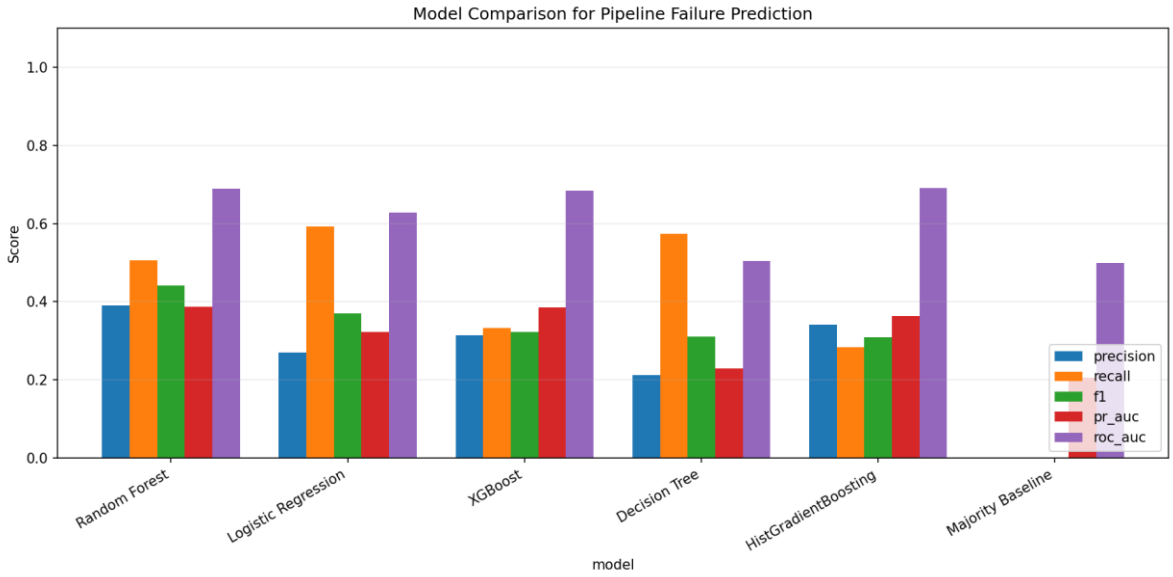

Figure 4.3: Model comparison chart for bounded Google 2019 sample

This was especially true for recall and F1-score, although the ETL/HDFS-compatible model did well on all held-out measures. For a failure predictor, good recall is desirable, since a failure is more damaging than a false positive alert. However, high recall must still be balanced against precision, as frequent false positives will decrease operator confidence in the system. For this reason, the presented F1-score is more indicative of model success than accuracy alone.

In a similar vein, the Alibaba and Google benchmark results represent a more conservative model picture. Their performance compared to the others reflects the greater difficulty of a production workload compared to the controlled ETL environment and prevents the report from claiming one good number that does not generalize.

This evidence from classifier comparison further demonstrates that the appropriate model may vary by source; XGBoost performed well for the developed live demo, while histogram gradient boosting performed best on the bounded Alibaba and random forest performed best on the bounded Google. This justifies an adaptive model-selection strategy that varies by data source, rather than the implementation of an overly strict heuristic.

The relatively good recall achieved by the ETL/HDFS-compatible model is indicative of what is expected from the system since the purpose of this project is the proactive identification of at-risk runs. In a monitoring setting, a missing failure is clearly more detrimental than a false positive. However, recall must not be examined in isolation; precision is required so that the

operators may trust the alerts and not dismiss a given prompt with the justification of it being merely another false positive. Hence, the F1-score shows a more balanced outcome for the failure class.

The additional measures ROC-AUC and PR-AUC further help explain how the models perform. ROC-AUC is the ability of a classifier to rank positive and negative examples appropriately across different thresholds. PR-AUC is most useful when considering less prevalent positive classes and thus was chosen to be preserved within the project artifact since the objective is to test how well the system works in the presence of class imbalance.

The Alibaba and Google scores were useful in precisely this: their weaker performance against those provided by the controlled ETL model is an indication of the variance in workload traces that are found, and that models often need to be tuned per source. This strengthens the report by showing a more cautious approach rather than hiding difficult evidence by choosing one good number that cannot always be generalized.

4.4 Interpretation of the Evaluation

The ETL/HDFS-compatible model had the highest results, as the demonstration traces are controlled and designed around visible phases, thus appropriate for a defense demonstration where the aim is to visualize the concept of end-to-end monitoring. However, this result is qualified by the fact that controlled scenarios are easier than uncontrolled, real-time traces.

The Alibaba result shows how metric accuracy can be misleading: class-imbalance results in a model identifying many of the successful jobs as success but fail to predict failures. Similarly, the Google 2019 cluster-trace result indicates that there may be model families which are better suited to different types of traces: random forest returned the best F1 score in the Google comparison while histogram-gradient boosting returned best results in the Alibaba traces.

The ETL/HDFS-compatible F1-score of 0.9143 also compares favorably with the DeepLog study's reported top-1 prediction accuracy of 88.9%, although the metrics and datasets are not identical (Du et al., 2017). The comparison is useful because both studies show that log-derived evidence can reveal abnormal execution patterns, but the present project adds an ETL monitoring dashboard, report output, and review workflow.

The bounded Alibaba and Google results also align with the caution raised by job-failure

prediction research: performance can vary by workload source, label imbalance, and feature schema (Jassas & Mahmoud, 2022). This explains why the report treats XGBoost as the selected implemented model while still recommending source-specific model comparison when the system is extended.

The overall result must therefore be interpreted cautiously. This study uses XGBoost as the implemented predictor and demonstrate it in a pipeline dashboard, but based on the comparisons, when extended, the model needs to remain source-dependent on different data sources.

The results must be interpreted in the context of what this project aims to do; that is, to build a prototype for visualizing ML-based predictive failure detection, rather than to provide a single optimal model that works for all cluster traces. Therefore, while the ETL/HDFS-compatible result shows that the proposed end-to-end system technically works, the benchmark results clearly indicate that the study has not provided a model that should be directly deployed in production.

However, the ETL controlled result still provides the value of proving the end-to-end flow of obtaining information from logs, converting into features, training and evaluating prediction model, and then reviewing the classification and generating reports. At the same time, the production-trace bounded results serve to remind the reader that additional datasets will have negative effect on the models: Therefore, the interpretation of the result must be balanced, not over-emphasized; the proposed system is feasible but still needs more testing in practice.

The comparison also raises awareness about solely using accuracy; in the imbalanced nature of predicting failure cases, a model may obtain high accuracy by correctly predicting the vast number of successful cases and ignoring failed ones, which demonstrates why recall, precision, F1, PR-AUC and MCC are much better performance metrics to analyze the question about failures. The same reasoning applies to the metric choices as described in the method section.

The evaluation also highlights the difference between demonstration and generalization validity; Demonstration validity indicates whether the built system is able to acquire data, extract features, predict, classify, generate review and reports. Generalization validity determines whether the same model would maintain the performance on different real-world pipelines. The current work provides evidence for the first question and preliminary results on the second.

The chosen experimental method supports the current research approach, where the system is mainly used to illustrate a concept of machine learning integration into pipeline monitoring as an end-to-end system, not to provide alternative to commercial enterprise observability platforms. As such, the results obtained show evidence that our developed prototype effectively enables an end-to-end ML-based monitoring workflow. On the other hand, the study does not demonstrate that the deployed thresholds or model files are to be directly deployed in each specific production environment unchanged.

4.5 Dashboard Analysis

The dashboard is structured around run control, live monitoring, logs, resources (metrics), comparing models, reports, and system view. These tabs provide a comprehensible flow for a supervisor or an examiner, because it is possible to observe the final prediction, as well as the stages within the pipeline, the logs, evidence of the risk and the report generated.

The sidebar allows the user to set the demo mode, scenario, minimum number of lines until the prediction is produced, policy behavior and playback controls. While in attached command mode, the user can provide a command that the runner will execute. The dashboard represents the predictor plugin as something that can be added next to a pipeline, rather than a static classifier.

The HIGH-risk review panel is a vital component of one of the defense mechanisms. When HIGH risk is found, when in the review mode, the process that triggered HIGH risk is exposed to the user and the operator has a choice whether to proceed or halt. This directly corresponds with the requirement in the project, stating that a HIGH risk should not just silently block the pipeline without presenting any rational choice for the operator.

Table 4.6: Dashboard components and evidence value

Component	What it displays	Why it matters
Run Control	Mode, scenario, policy, seed, loop and command options.	Shows that the demo is configurable.
Stage Board	Extract, validate, transform, load and report stages.	Makes the pipeline visually recognizable.
Log Stream	Recent generated or attached log lines.	Shows evidence behind predictions.
Risk KPIs	Current risk, counts, HIGH-risk status and explanations.	Connects model output to operator decision-making.
Review Controls	Continue and stop options for HIGH risk.	Demonstrates intervention workflow.
Model Comparison	Saved charts and comparison tables.	Supports the algorithm comparison objective.

Component	What it displays	Why it matters
Report Tab	JSON payload and generated report paths.	Supports post-run auditability.
System Map	High-level architecture of modules.	Helps explain the implementation during defence.

It is on the dashboard that we most clearly see that there is an operator-facing workflow to the project. Here, the user can select an operating mode and scenario within the run-control area, and the pipeline progress is visualized in the monitoring panels. This makes the system interactive and inspectable. Interactive and inspectable systems are necessary because it is this interactivity which enables the examiner to trace the pipeline through its stages.

The log stream and stage board serve to bridge the gap between the prediction and observable phenomena. The system's transition into a HIGH-risk state is easier to grasp when displayed next to relevant warnings and exceptions such as the warning lines, failing validation messages, retries or delays in stage execution. This makes the model less of a black box: an operator can clearly see the operative signals that accompany a particular prediction.

Review controls provide another interesting outcome: the project clearly does not consider every model prediction to be an executable command in all operating circumstances. The pipeline can enter a state in which it waits for user intervention; it can continue after that or shut down, based on predefined policy. For an operational system this is crucial, because while shutdown can help prevent a data quality catastrophe, it may also cause disruption to subsequent business processes.

The model-comparison and report sections extend the view provided at the dashboard: here the operator can examine historical records and generate reports on system operations. This provides a crucial link in the chain of accountability because such records allow for post-mortem review after a dashboard session has terminated.

4.6 Airflow Orchestration Evidence

The orchestration evidence here takes the form of airflow not as a dashboard, but as an orchestration tool. The DAG named pipeline_failure_monitor_demo has no schedule by default, which makes it suitable for manual defence triggering. It includes parameters for demo_type, pipeline_command, scenario, loops, min_lines, stop_policy, and fail_airflow_task_on_high.

The DAG tasks are prepare_run, run_pipeline_with_predictor, evaluate_intervention, and publish_report. This is evidence that the project workflow can be framed as an ordered sequence

of pipeline tasks. The evaluate_intervention task, by checking HIGH-risk counts, can throw an Airflow exception if it is configured to fail the orchestration task on HIGH risk.

This evidence should be described carefully. Although Airflow is part of the project evidence and can run a pipeline, the actual Render deployment does not have an Airflow server running.

The Airflow DAG shows that monitoring logic can be mapped to an Airflow workflow of pipeline tasks. This is significant as typical data pipelines are orchestrated, not just executed by a single command. By defining preparation, monitored execution, intervention evaluation, and report publishing as tasks, the project shows how the predictor can fit into a broader workflow model.

The Airflow DAG parameters, demo type, pipeline command, scenario, loops, minimum lines, stop policy and failure-on-high behavior allow for different runs to be executed. The flexibility of parameters makes the Airflow DAG more than simply static evidence.

This is a reserved inference. It is important to clarify that it's only to be considered local orchestration evidence; on Render, it only shows a dashboard running. Actual implementation with Airflow would need work related to persistence, security, workers and infrastructure..

4.7 Report Generation Evidence

The report generator, which generates the JSON and HTML reports. The JSON report will store the evidence of monitoring in structured way. HTML report can be read by an operator and displays operator facing summary cards, risk count, high-risk entries, alert row, log row, and resources summary. This is a good project because monitoring will not finish once the visual session finishes.

The predictor plugin report payload contains the status of session, the stop policy, current risk, the count of risks, alerts, high-risk items, log records, stage summary, resources summary and decision metadata. This will help to trace, and it also provides the way for the examiner to examine what went on after the execution.

Table 4.7: Report payload evidence

Report element	Evidence recorded	Usefulness
Session summary	Status, current risk, counts and policy.	Explains the run outcome.
Alerts	Risk level, probability, explanation and timestamp.	Preserves prediction history.

Report element	Evidence recorded	Usefulness
High-risk items	Filtered HIGH-risk alerts.	Supports intervention review.
Logs	Recent emitted or adapted messages.	Links decisions to process evidence.
Resource summary	CPU, memory, disk and backend metadata.	Adds runtime context.
Report paths	JSON and HTML file locations.	Supports post-run inspection.

Report generation is a critical function because operational monitoring ought to produce some artifacts to be reviewed. A session viewed on a dashboard exists only as long as it is being viewed. A JSON or HTML report generated at the end of a session, however, can preserve an account of what happened during that session. This allows for a system of accountability and post-run diagnosis of the model's or pipeline's execution.

The JSON report is especially useful for machine readable review because it saves the specific values of individual fields, to be loaded and inspected programmatically at some future time. Similarly, the HTML report is useful for human review, because it displays the session's overall status, count of risks, critical high-risks, logs, and resource summary, in an easily understandable format, to serve as either evidence, or as the artifact itself. These two forms together allow for machine-readable and operator-readable evidence of the model or pipeline's run.

Finally, the report payload is essential because it provides the link between the prediction and the intervention choice. By storing information related to the alerts that cause a HIGH-risk assessment, and the policy logic invoked, the distinction between model score, operator's assessment, and pipeline output, becomes clearer.

4.8 Deployment Readiness Analysis

The analysis of deployment preparedness focuses on the Streamlit dashboard. The repository has been updated to include a render.yaml and a ..python-version. A Render start command runs the Streamlit dashboard at 0.0.0.0 and uses the render specified port, facilitating a web version of the visual dashboard.

The deployment itself has deliberately been designed to be as small as possible and has been prepared for web hosting, hence excluding all large raw datasets, installer files, generated reports, Word documents, and research logs. The web service is composed of trained model artifacts, small example datasets, and saved comparisons. This is appropriate as we are concerned with hosting a visual dashboard demonstration in the cloud and not with a gigabyte-

sized trace upload during a cloud demonstration.

The description of the project as a prototype deployment should still be maintained. The Render deployment demonstrates that it is possible to host the visual dashboard as a web service but does not in any way imply it is ready for production, continuous retraining, security features such as authentication, multi-user role-based access control or long-term data storage.

The analysis of deployment readiness has been prepared from the perspective of a lightweight defense demonstration; there are appropriate files in the repository to deploy Streamlit as a service on Render. These files are a requirements.txt which defines the required libraries and a .python-version which defines the python environment, allowing for the visual dashboard to be deployed without the examiner having to install the full local system.

The analysis also includes the file requirements for the Render deployment. The raw public traces, generated reports, installer files, and local Word documents were excluded to keep the deployment manageable; the appropriate demonstration deployment includes the source code, trained models, example data, and saved charts. This conservative approach also relates to broader data-warehousing concerns of the necessity for cloud readiness, scalability and maintainable data architectures (Srikanth & Vishnu, 2024).

The product should not be presented as a fully deployed enterprise service. A production service would require authentication and authorization, secure handling of sensitive information such as secrets and keys, long-term persistent storage, persistent logging, support for a model registry, continuous observation and monitoring of performance, sophisticated handling of errors and infrastructure level monitoring. This ensures that demonstration deployment isn't oversold.

The inclusion of a description of what further would be needed if this were an enterprise product provides useful additional commentary on what would be required beyond the scope of this project. A fully developed product would require a secure handling of logs, authentication for users to the service, permanent storage of reports generated by the system and audit trails. A continuous model observation facility, persistent logging of operations, model artifact management, system level monitoring and an acceptable model update protocol.

4.9 Summary of Findings

The findings indicate that the project advanced from a terminal-based predictor to a visual

monitoring prototype including real-time ETL running, plug-in based data ingestion, risk classification, high risk intervention controls, report generating and evidence of model comparing and evidence of Airflow orchestration. What is of most technical value is the workflow as opposed to any one score. The greatest limitation is that production-scale verification will occur in the future.

The findings demonstrate that the project provided practical integration between model evidence and system activity. The implemented workflow allows us to execute a pipeline, gather logs, update risk, display intervention controls and create reports. The evaluation metrics demonstrated efficient and controlled ETL performance and only acceptable benchmark performance, indicating that the prototype can be evaluated as providing reasonable overall functionality.

The most significant contribution is the concept of end-to-end monitoring, which offers failure prediction as a plugin-based workflow instead of simply a standalone classifier, enabling observation, decision-making, and auditability. A primary weakness, on the other hand, is the lack of full production-scale verification and multi-user capability.

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary

The project developed a log analysis and failure prediction system for data pipeline batch jobs using XGBoost and supporting machine learning comparisons. The system incorporates loaders, extractors, model training, model comparison, a predictor plugin, a live ETL runner, Streamlit dashboard, reports, and an Airflow DAG.

The central improvement over the earlier implementation is the visual and operational workflow. The user can kick off a pipeline, watch stages and logs, look at dynamically changing risk classifications, view HIGH-risk cases, and resume or stop it, while also generating a report. This makes the project defendable because it shows how the model would fit alongside a real pipeline.

The study addressed the problem of detecting and presenting failure risk in data pipeline batch jobs. It combined machine-learning model development with a software prototype that can observe logs, extract features, classify risk, display evidence, and generate reports. This combination is important because failure prediction is useful only when the prediction reaches an operator in a form that can guide action.

The project increases the defensibility of the work by segregating evidence layers. These layers include model evidence from offline training and comparisons, evidence from the ETL demonstration, reusable behavior evidence from the predictor plugin, visual evidence from the Streamlit dashboard, and orchestration evidence from the Airflow DAG. These different pieces are more useful to defense than an isolated notebook or static interface.

The results show that the controlled ETL/HDFS-compatible model performed strongly, while the bounds of the Alibaba and Google benchmark comparisons present some difficulty. This mixed result is desirable as it suggests the prototype is promising but requires more widespread testing before definitive claims about generalization can be made.

5.2 Conclusion by Objectives

Table 5.1: Objective-level conclusion

Objective	Conclusion
Identify failure patterns	The feature engineering layer captures severity counts, failure keywords, retry behavior, task failures, scheduling delay and resource context.
Compare models	Saved comparison outputs show that XGBoost was evaluated beside logistic regression, decision tree, random forest, histogram gradient boosting and a baseline.
Develop monitoring predictor	The predictor plugin and PipelineMonitor connect trained artifacts to streaming logs and produce risk classifications.
Evaluate model	Stored metrics provide precision, recall, F1, ROC-AUC, PR-AUC and confusion-matrix evidence.
Visualize workflow	The Streamlit dashboard and Airflow DAG provide visual monitoring and orchestration evidence.

The first objective, which concerned the identification of failure patterns, was accomplished using feature engineering. The system extracts severity counts, error and warning rates, keywords for failures, presence of retribution, timing-based features, workload-structure features. All features can render raw operational evidence to a format, which can be consumed by supervised learning and live monitoring.

The second objective, which concerned model comparison, was accomplished using saved comparison output. As multiple classifiers were investigated, their results are saved for discussing selected implementation over others. As one cannot assume, a model is best without comparing other options.

The third objective, which concerned the development of a monitoring predictor, was accomplished using the predictor plugin, live runner, dashboard integration. The system can consume log evidence, update risk classification, store alerts, and show the status to the user. The fourth goal (evaluate) was accomplished using metrics like precision, recall, F1-score, ROC-AUC, PR-AUC, MCC, specificity, confusion matrix counts. The fifth goal (visualize) was accomplished using the Streamlit dashboard and Airflow workflow evidence.

Overall, the objectives were achieved in a chained manner rather than separated steps. The preparation steps make feature extraction possible; the feature extraction enables training a model; the comparison leads to a specific implementation choice; the predictor plugin links the model to real time logs; the dashboard shows the results. The chain of events has importance in demonstrating project progress and moving the project from data to decision support system.

The objective related to visualization was particularly important for the final project because it transformed the work from a technical model into a demonstrable system. The dashboard, report output, and Airflow DAG make the system easier to inspect and defend. They show not only what the model predicted, but also how the prediction fits into a recognizable data pipeline workflow.

5.3 Recommendations

- Keep the deployed Render service focused on the Streamlit dashboard and plugin demonstration. Airflow should remain orchestration evidence unless a separate Airflow deployment is required.
- Add a visible dashboard toggle that separates raw model probability from demonstration presentation percentage. This will make the honesty of the demonstration even clearer.
- Expand production-trace validation when more compute time is available. Larger Alibaba and Google samples should be evaluated in repeated runs before making stronger generalization claims.
- Introduce model-version tracking so that each report can identify the model artifact used for a monitoring session.
- Add authentication if the dashboard is used beyond defence demonstration, because pipeline logs and predictions can contain sensitive operational information.
- Add explainability tools such as SHAP only after the existing model pipeline is stable, and label such explanations as model explanations rather than causal proof.
- Calibrate LOW, MEDIUM and HIGH thresholds with user-defined cost assumptions before production deployment. A missed failure and a false alarm do not have equal operational cost.

The first recommendation is to treat the current system as a validated prototype rather than a finished enterprise platform. It is suitable for demonstration, research discussion, and further development, but a production deployment should include stronger operational controls such as authentication, persistent storage, model registry support, and structured logging.

The second recommendation is to continue separating raw model probability from demonstration presentation values. This distinction protects the academic honesty of the work. The dashboard can remain visually clear during defence while the report still preserves the raw model evidence used for evaluation.

The third recommendation is to expand validation before deployment in a real organization. Larger and repeated experiments should be carried out on additional traces, real pipeline logs, and different failure categories. This will help determine whether the thresholds, features, and selected model remain effective outside the controlled demonstration.

The fourth recommendation is to improve explainability only after the base monitoring workflow remains stable. Methods such as SHAP can help show which features influenced a prediction, but explanations should be presented as model explanations rather than proof of the true root cause. Root-cause confirmation still requires engineering investigation.

The system should be extended with stronger report storage. At present, reports can be generated for inspection, but a production-oriented version should store them in a database with session identifiers, timestamps, model versions, risk counts, operator decisions, and final job outcomes. This would support trend analysis and make it possible to compare predictions against later confirmed failures.

Another recommendation is to introduce model-version governance. Each monitoring report should record the model name, training date, feature schema, threshold configuration, and evaluation summary used during that session. This will make future audits easier because an engineer can determine exactly which model produced a particular warning.

The dashboard should also include clearer separation between operational controls and research evidence. Operational users need simple risk cards, alerts, and actions, while researchers and examiners may need metrics, comparison charts, and feature explanations. Separating these views would make the interface cleaner for each audience while preserving the evidence needed for academic review.

5.4 Limitations of the Study

This section discusses known limitations of the approach. The live ETL stream is an engineered demonstration pipeline, which reflects stages of known data engineering but does not fully

emulate the storage systems, orchestration queues, business constraints or dependencies of an enterprise.

The Alibaba and Google tests were local, bounded samples of the given traces. They serve as important evidence to the experiment, but not as full-trace production validation. The claim that the model has been proven across all workloads within those traces is avoided.

The dashboard can present an integer risk percent that varies during the demonstration. This is useful for a defense, to ensure the display looks dynamic, but the integer presented is not raw probability of failure.

The Render deployment packages the Streamlit dashboard as a web service. It does not package the orchestration service Airflow, background workers, persisted storage, user authentication, nor continuous retraining.

One limit of this experiment is that it is the live ETL demonstration. It represents recognizable data engineering phases with some real logs, but doesn't fully reflect dependencies, message queues, storage systems, authentication services, outside-of-the-program service failure, or concurrent workflows like one would see in an enterprise pipeline.

Another limitation is the bounded size of the test traces. While the Alibaba and Google test traces expand the evidence base over previous tests of the system, the tests do not run all available data from those traces. Future tests of the system need to explore further traces by running on these publicly available samples multiple times.

Another limit to this work is threshold tuning. The thresholds LOW, MEDIUM, HIGH were selected as intuitive and useful demonstration choices but would be business-dependent in practice and set by operational costs. For instance, one system might assign higher costs to failed alarms (and so low, medium, high thresholds would reflect higher numbers) than another where missing failure (high false negative) is a more cost critical issue.

Finally, the limits of the Render deployment should be mentioned. The system only exposes the Streamlit dashboard; there is no deployment of Airflow workers, persistence, or user authentication in this test. This implies the online test should be thought of as showing off a web accessible prototype and nothing more.

It would be useful to have a long-term history view. The prototype can already generate a report

showing the stream at any given moment in time but does not yet analyze longer histories over days or months, in order to detect seasonality, recurring pipeline problems, or drift.

One final missing item is an official explanation of the prediction being displayed on the Streamlit dashboard. The feature evidence and risk summary explains the reasoning, but none of the methods like SHAP are currently employed at the model display time to better represent reasoning, but could be later, making for an easier defense of the prototype.

Another limit is operational integration. Although the system shows attachment to a foreign command on the live stream and the Airflow DAG does display the command executed, the actual integration points with other enterprise systems are missing such as schedulers, data warehouses, incident managers, etc., and need to be developed for use in a production system.

5.5 Suggestions for Further Work

Future work should package the predictor as a formal service or installable plugin with a documented API, configuration file, distribution package, and versioned model artifacts. This would make it easier for other pipelines to call the monitor without manually copying project scripts.

It would also be useful in the future version of the monitor to add persistent storage to monitoring sessions. By logging reports to a database, it would be possible to perform trend analysis and threshold tuning, and to defend model predictions with repeated defenses.

An additional, future feature would be support for multiple concurrent pipelines. The present dashboard is best viewed as a demonstration of a single-session monitor; a real-time operations console would need session isolation, user permissions, queueing, and alert routing.

Other sequence-aware log models could also be tested against the current engineered-feature approach to monitor performance, but care must be taken when evaluating more complex models.

The system should be tested on real organizational pipeline logs following a resolution of privacy and access issues, and to determine if the engineered-feature method is resilient to business-specific failure modes, missing fields, operational noise, and inconsistent logging.

5.6 Final Conclusion

This project delivers a defensible final year prototype for analyzing logs and predicting failures in data pipeline batch jobs. Its main contribution is linking machine learning evidence with an understandable operational workflow: A pipeline runs, logs are observed, features extracted, failure risk assessed and classified. A HIGH-risk level flags it for investigation and report generation. While not presented as a fully finished production system, it is a sound starting point for a monitoring plugin which is capable of further enhancement and evaluation.

To conclude, this project succeeds in presenting a defensible prototype that can be used for log analysis and failure prediction for data pipeline batch jobs. Its integration of supervised learning, source-specific model artifacts, real-time ETL monitoring, visualization through a dashboard, HIGH risk intervention logic, report generation, and orchestration evidence is its greatest achievement. The success in both demonstrating prediction ability and communicating the prediction via a practical monitoring workflow directly satisfies the aims of the project.

The success of this project should be considered a building block for further research. Its primary academic contribution lies in the coherent integration of model evidence and operational workflow, whereas its key limitation for production is the need for more validation and hardening. With increased data, refined thresholds, data persistence, access control and integrated plugin packaging, the system can be developed into a valuable tool for proactively ensuring data-pipeline reliability.

The final delivered product also highlights that a final-year project can be both conceptually well researched and functionally demonstrative. The literature review demonstrates why pipeline failure prediction is an important field, while the methodology discusses how to acquire and process the data and models used. Evaluation and output are presented in the results chapter, and limitations are addressed in the conclusion. Together, all these chapters tell a consistent story about the necessity for early prediction and the presentation of this prediction in a usable form.

The key takeaway from the project is that prediction alone is insufficient. Effective monitoring requires integrating data preparation, feature engineering, model evaluation, real-time data streams, interpretation of predicted risk, an intervention strategy, and final reporting. The demonstrated prototype clearly connects all these components. While still requiring significant testing and production integration, it serves as a strong basis for future advancements in intelligent pipeline monitoring.

This project also contributes by establishing a visible link between automation and human decision-making. While the model can estimate failure risk, the operator can still review and make a judgment based on available evidence. By correlating logs, stages, alerts, resource samples, and reports in a single view, it avoids automation 'black box' operations and instead guides informed interpretation. This is an important characteristic for data engineering, as the decisions to stop, proceed, or escalate to another engineer have business implications.

In a practical sense, the prototype illustrates how batch-job reliability can be improved without organizations needing to change their existing pipelines. The attachment mechanism to an ongoing pipeline allows a predictor to monitor an external process, use the observed data as input features for the model, classify its risk and then retain the entire session for later inspection. The extensible nature of the prototype means that in future versions the sample ETL job can be replaced by specific organizational commands and reports integrated with data stores and thresholds tuned against live incident data.

For academic purposes, the project is defensible. The contribution has been identified as the integration of predictive modeling and a visualizable workflow that incorporates monitoring and intervention. The main limitation has also been identified as the need for further testing, greater security controls, a robust persistent data store and corporate integration prior to production use. The conclusion therefore accurately reflects the state of the work.

REFERENCES

Akash, V. C., & Charate, P. C. (2025). Proactive data pipeline maintenance via machine learning-driven anomaly detection. International Journal of Scientific Research in Science and Technology, 12(2), 1041-1053. https://doi.org/10.32628/ijsrst251222663

Alibaba Group. (2022). Cluster trace GPU v2020 [Data set]. GitHub. https://github.com/alibaba/clusterdata/tree/master/cluster-trace-gpu-v2020

Apache Software Foundation. (n.d.). DAGs. Apache Airflow documentation. Retrieved June 2, 2026, from https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html

Arif, A. Z., Abduljabbar, H. Z., Tahir, A. H., Bibo, S. A., & Almufti, S. M. (2023). eXtreme gradient boosting algorithm with machine learning: A review. Academic Journal of Nawroz University, 12(2), 320-334. https://doi.org/10.25007/ajnu.v12n2a1612

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32. https://doi.org/10.1023/A:1010933404324

Chanda, D. (2024). Automated ETL pipelines for modern data warehousing: Architectures, challenges, and emerging solutions. The Eastasouth Journal of Information System and Computer Science, 1(3), 209-212. https://doi.org/10.58812/esiscs.v1i03

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785-794). https://doi.org/10.1145/2939672.2939785

Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273-297. https://doi.org/10.1007/BF00994018

Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (pp. 1285-1298). https://doi.org/10.1145/3133956.3134015

Foidl, H., Golendukhina, V., Ramler, R., & Felderer, M. (2024). Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for

developers. Journal of Systems and Software, 207, Article 111855. https://doi.org/10.1016/j.jss.2023.111855

Google. (2019). Borg cluster workload traces from 2019 [Data set documentation]. GitHub. Retrieved June 2, 2026, from https://github.com/google/cluster-data/blob/master/ClusterData2019.md

Informatica. (2025). What is an ETL pipeline? https://www.informatica.com/resources/articles/what-is-etl-pipeline.html

Jassas, M. S., & Mahmoud, Q. H. (2022). Analysis of job failure and prediction model for cloud computing using machine learning. Sensors, 22(5), Article 2035. https://doi.org/10.3390/s22052035

Ma, X., Li, Y., Keung, J., Yu, X., Zou, H., Yang, Z., Sarro, F., & Barr, E. T. (2024). Practitioners' expectations on log anomaly detection. arXiv. https://arxiv.org/abs/2412.01066

Maalouf, M. (2011). Logistic regression in data analysis: An overview. International Journal of Data Analysis Techniques and Strategy, 3(3), 281-299.

Matzavela, V., & Alepis, E. (2021). Decision tree learning through a predictive model for student academic performance in intelligent m-learning environments. Computers and Education: Artificial Intelligence, 2, Article 100035. https://doi.org/10.1016/j.caeai.2021.100035

Mienye, I. D., & Jere, N. (2024). A survey of decision trees: Concepts, algorithms, and applications. IEEE Access, 12, 86716-86727. https://doi.org/10.1109/ACCESS.2024.3416838

Naeem, M., Rizvi, S., & Coronato, A. (2020). A gentle introduction to reinforcement learning and its application in different fields. IEEE Access, 8, 209320-209344. https://doi.org/10.1109/ACCESS.2020.3038605

Naeem, S., Ali, A., Anam, S., & Ahmed, M. M. (2023). Unsupervised machine learning algorithms: Comprehensive review. International Journal of Computing and Digital Systems, 13(1), 911-921. https://doi.org/10.12785/ijcds/130172

Nishanth R. M. (2019). The evolution of ETL architecture: From traditional data warehousing to

real-time data integration. World Journal of Advanced Research and Reviews, 1(3), 073-084. https://doi.org/10.30574/wjarr.2019.1.3.0033

Ogunsola, K. O., Balogun, E. D., & Ogunmokun, A. S. (2022). Developing an automated ETL pipeline model for enhanced data quality and governance in analytics. International Journal of Multidisciplinary Research and Growth Evaluation, 3(1), 791-796. https://doi.org/10.54660/.ijmrge.2022.3.1.791-796

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830. https://www.jmlr.org/papers/v12/pedregosa11a.html

Popovic, A., Ivkovic, V., Trajkovic, N., & Lukovic, I. (2024). A domain-specific language for managing ETL processes. PeerJ Computer Science, 10, Article e1835. https://doi.org/10.7717/peerj-cs.1835

Quinlan, J. R. (1986). Induction of decision trees. Machine Learning, 1(1), 81-106. https://doi.org/10.1007/BF00116251

Reddy, P., Viswanath, P., & Reddy, B. E. (2018). Semi-supervised learning: A brief review. International Journal of Engineering & Technology, 7(1), 81-85.

scikit-learn developers. (n.d.-a). StratifiedGroupKFold. Scikit-learn documentation. Retrieved June 2, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedGroupKFold.html

scikit-learn developers. (n.d.-b). f1_score. Scikit-learn documentation. Retrieved June 2, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html

Seenivasan, D. (2025). AI-driven enhancement of ETL workflows for scalable and efficient cloud data engineering. International Journal of Engineering and Computer Science, 13(20), 26837-26848. https://doi.org/10.18535/ijecs/v14i02.4824

Shaveta. (2023). A review on machine learning. International Journal of Science and Research Archive, 9(1), 281-285. https://doi.org/10.30574/ijsra.2023.9.1.0410

Srikanth, G., & Vishnu, C. (2024). The future of data warehousing: Trends, technologies, and challenges in the era of big data, cloud computing, and artificial intelligence. International Journal of Scientific Research in Computer Science, Engineering and Information Technology, 10(5), 470-479. https://doi.org/10.32628/cseit241051029

Streamlit. (n.d.). st.fragment. Streamlit documentation. Retrieved June 2, 2026, from https://docs.streamlit.io/develop/api-reference/execution-flow/st.fragment

Suyal, M., & Goyal, P. (2022). A review on analysis of k-nearest neighbor classification machine learning algorithms based on supervised learning. International Journal of Engineering Trends and Technology, 70(7), 43-48. https://doi.org/10.14445/22315381/IJETT-V70I7P205

Udousoro, I. C. (2020). Machine learning: A review. Semiconductor Science and Information Devices, 2(2), 5-14. https://doi.org/10.30564/ssid.v2i2.1931

Wang, Y., Mäntylä, M. V., Nyyssölä, J., Ping, K., & Wang, L. (2024). Cross-system software log-based anomaly detection using meta-learning. arXiv. https://arxiv.org/abs/2412.15445

Weng, Q., Xiao, L., Yu, C., Wang, W., Wang, C., He, J., Li, Y., Zhang, L., & Ding, W. (2022). MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters. In 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22) (pp. 945-960). https://www.usenix.org/conference/nsdi22/presentation/weng

Yang, L., Chen, J., Wang, Z., Wang, W., Jiang, J., Dong, X., & Zhang, W. (2021). Semi-supervised log-based anomaly detection via probabilistic label estimation. In Proceedings of the International Conference on Software Engineering (pp. 1448-1460). https://doi.org/10.1109/ICSE43902.2021.00130

Yuan, Y., Wu, Y., Wang, Q., Yang, G., & Zheng, W. (2012). Job failures in high performance computing systems: A large-scale empirical study. Computers and Mathematics with Applications, 63(2), 365-377. https://doi.org/10.1016/j.camwa.2011.07.040

Zhang, L., Jia, T., Jia, M., Li, Y., Yang, Y., & Wu, Z. (2024). Multivariate log-based anomaly detection for distributed database. arXiv. https://arxiv.org/abs/2406.07976

Zhu, J., He, S., Liu, P., He, Q., Xu, Y., & Lyu, M. R. (2023). Loghub: A large collection of system log datasets for AI-driven log analytics. In 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE) (pp. 355-366). https://doi.org/10.1109/ISSRE59848.2023.00071

Zou, J., & Petrosian, O. (2020). Explainable AI: Using Shapely value to explain complex anomaly detection ML-based systems. ResearchGate. https://www.researchgate.net/figure/DeepLog-architecture_fig1_347324639